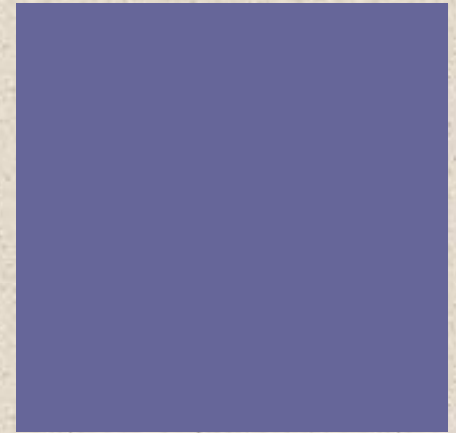
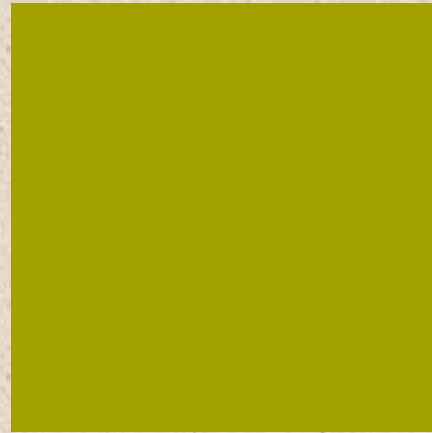
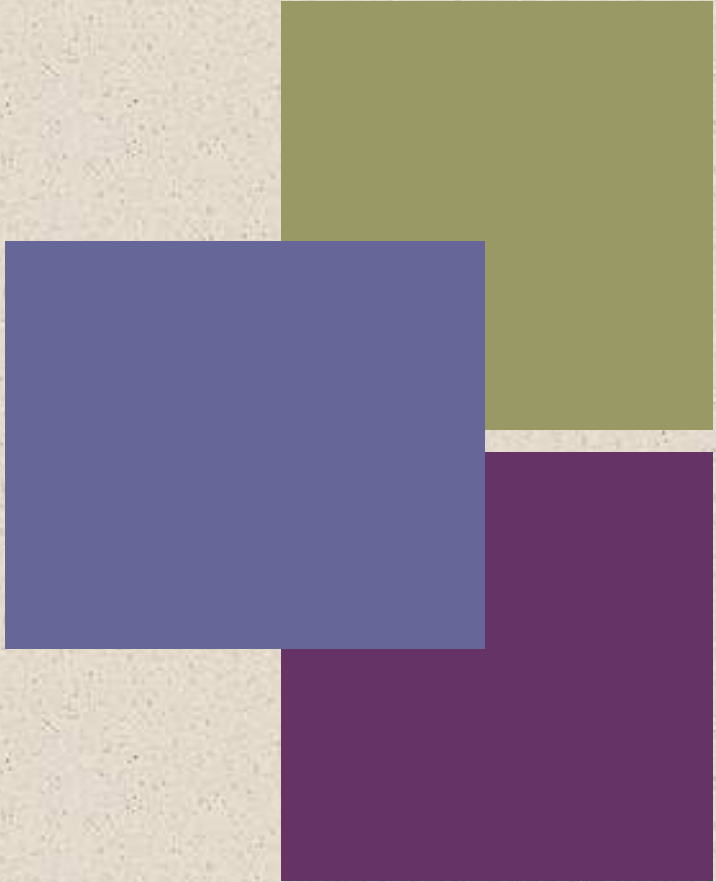




William Stallings Computer Organization and Architecture 10th Edition



+ Chapter 3

A Top-Level View of Computer Function and Interconnection



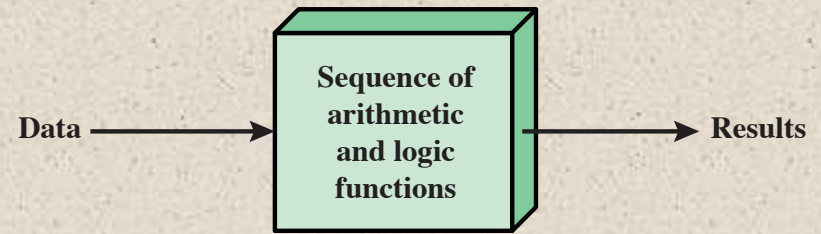
Computer Components



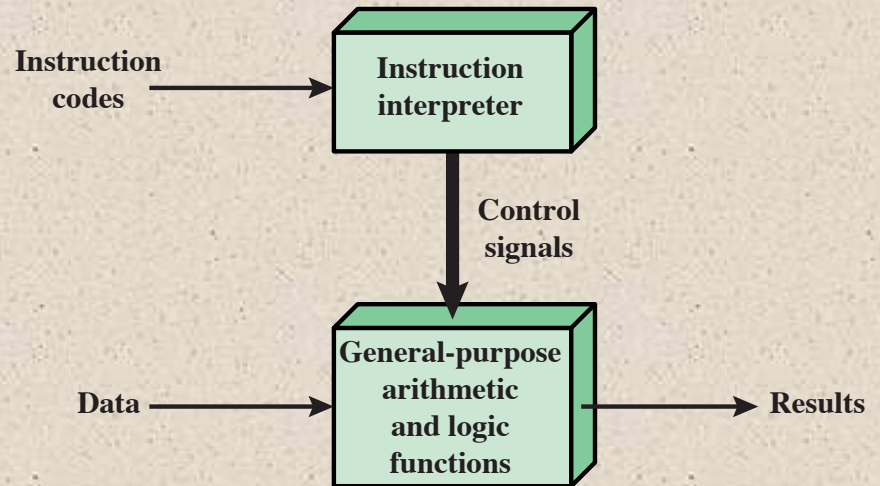
- Contemporary computer designs are based on concepts developed by John von Neumann at the Institute for Advanced Studies, Princeton
- Referred to as the *von Neumann architecture* and is based on three key concepts:
 - Data and instructions are stored in a single read-write memory
 - The contents of this memory are addressable by location, without regard to the type of data contained there
 - Execution occurs in a sequential fashion (unless explicitly modified) from one instruction to the next
- *Hardwired program*
 - The result of the process of connecting the various components in the desired configuration



Hardware and Software Approaches



(a) Programming in hardware



(b) Programming in software

Figure 3.1 Hardware and Software Approaches

Software

- A sequence of codes or instructions
- Part of the hardware interprets each instruction and generates control signals
- Provide a new sequence of codes for each new program instead of rewiring the hardware

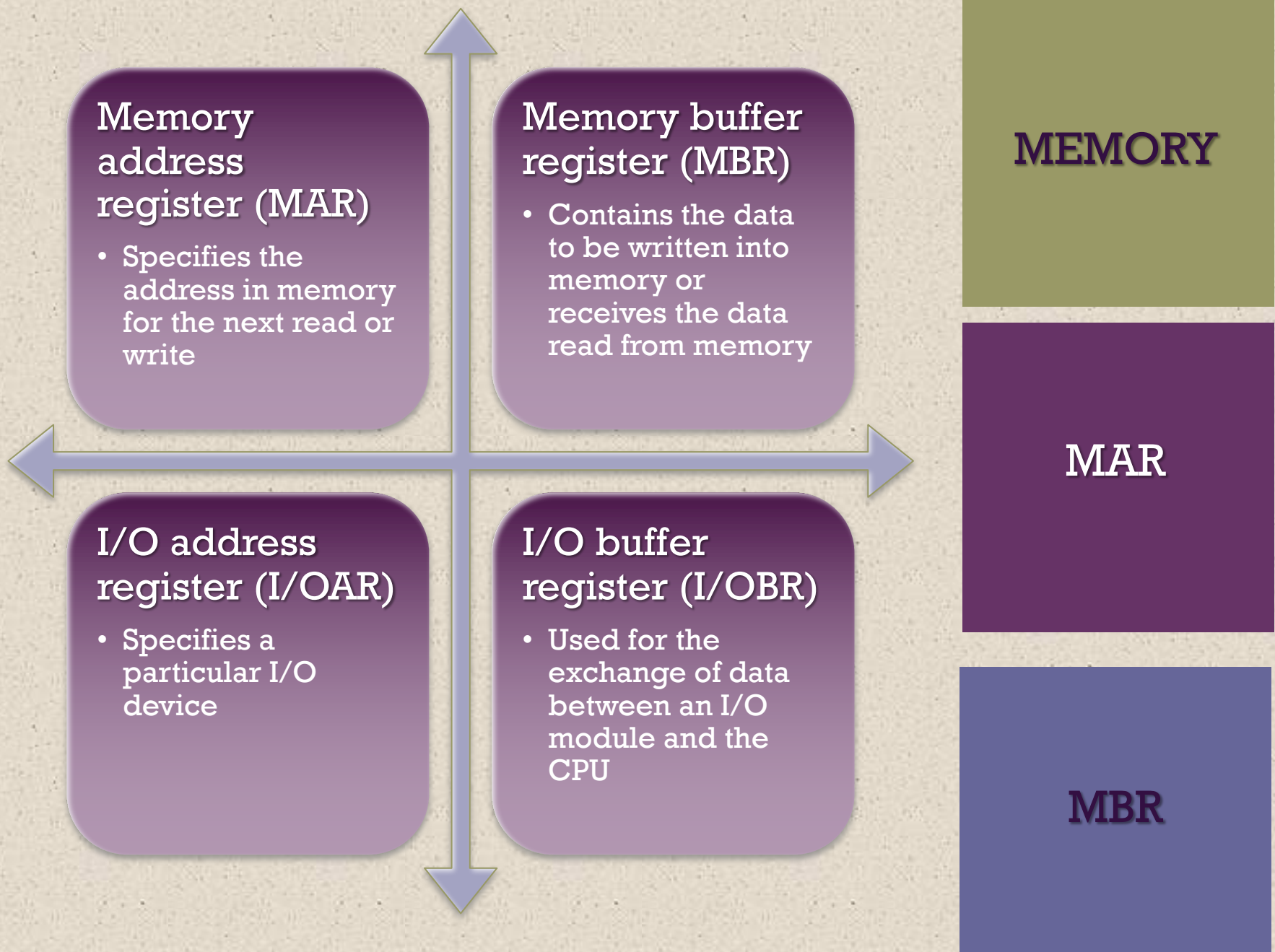
Major components:

- CPU
 - Instruction interpreter
 - Module of general-purpose arithmetic and logic functions
- I/O Components
 - Input module
 - Contains basic components for accepting data and instructions and converting them into an internal form of signals usable by the system
 - Output module
 - Means of reporting results

Software

I/O
Components





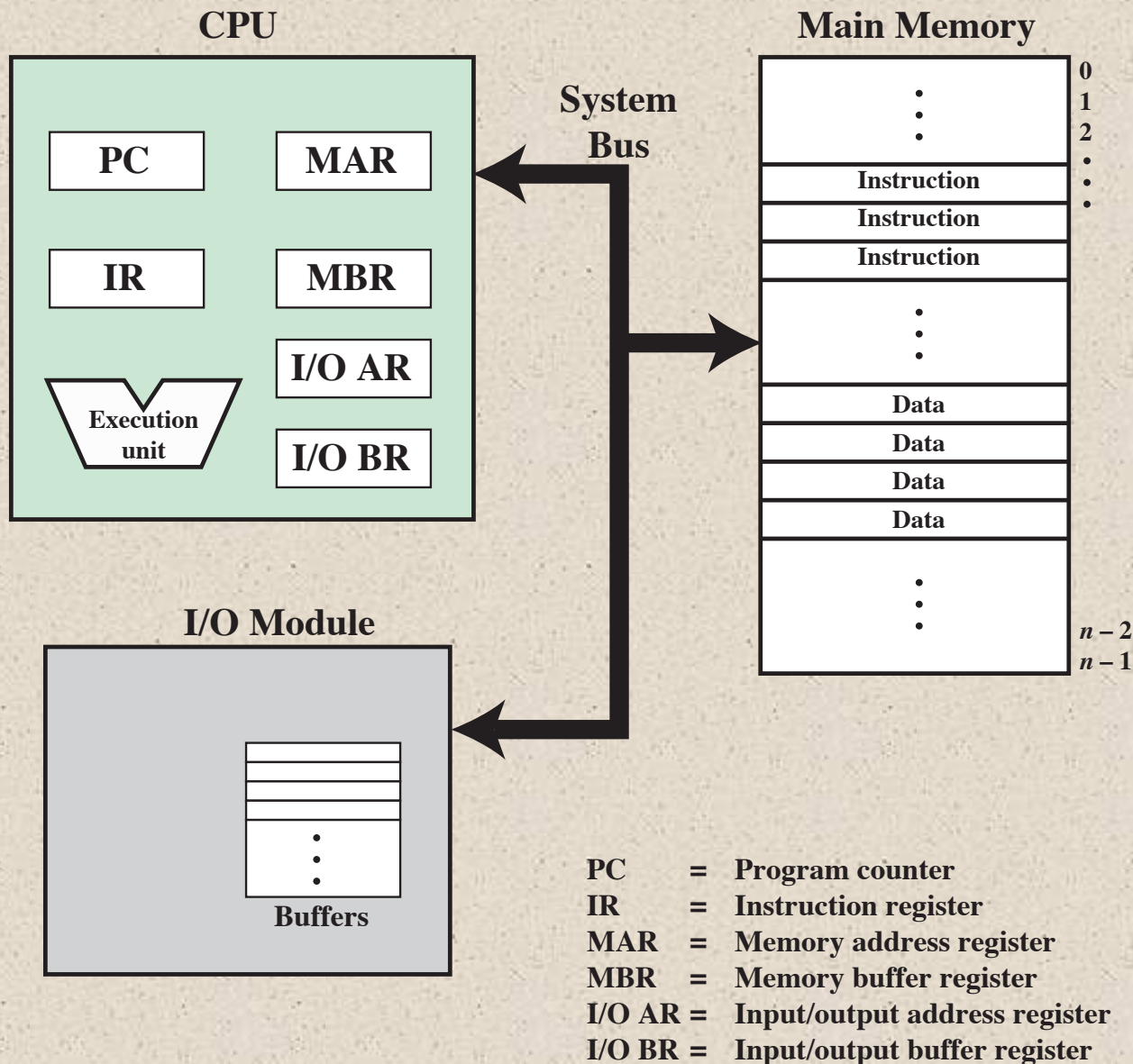


Figure 3.2 Computer Components: Top-Level View

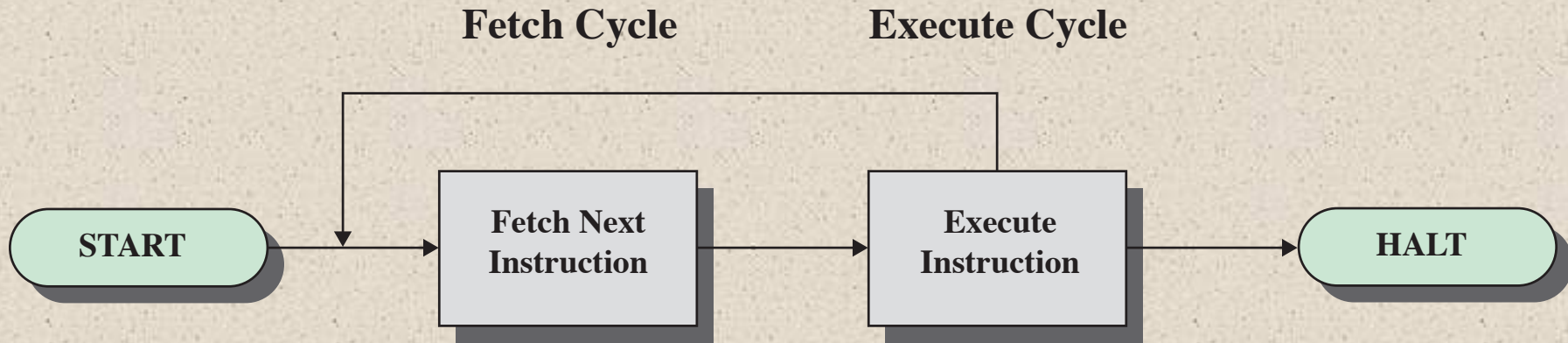


Figure 3.3 Basic Instruction Cycle



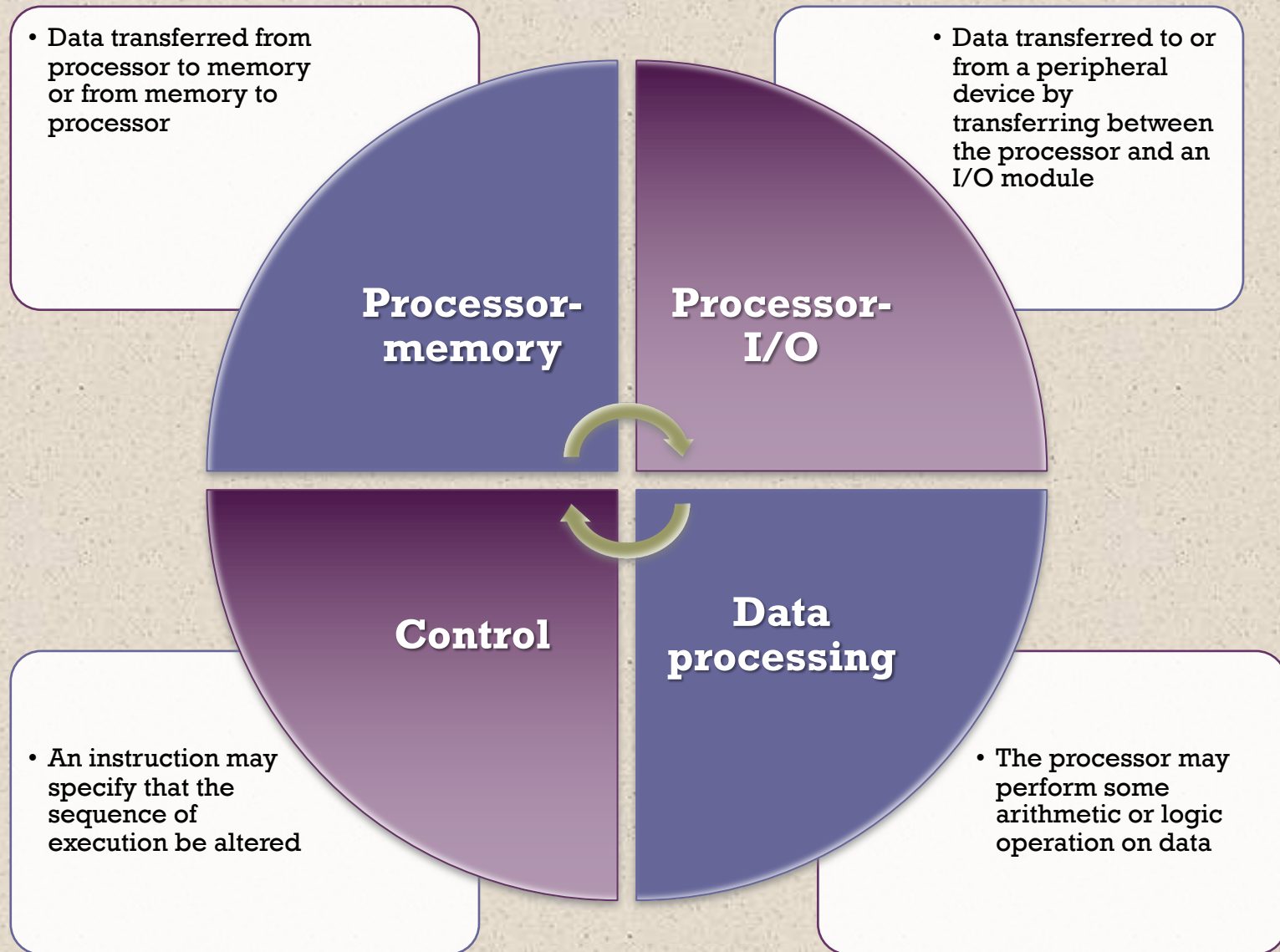
Fetch Cycle



- At the beginning of each instruction cycle the processor fetches an instruction from memory
- The program counter (PC) holds the address of the instruction to be fetched next
- The processor increments the PC after each instruction fetch so that it will fetch the next instruction in sequence
- The fetched instruction is loaded into the instruction register (IR)
- The processor interprets the instruction and performs the required action



Action Categories





(a) Instruction format



(b) Integer format

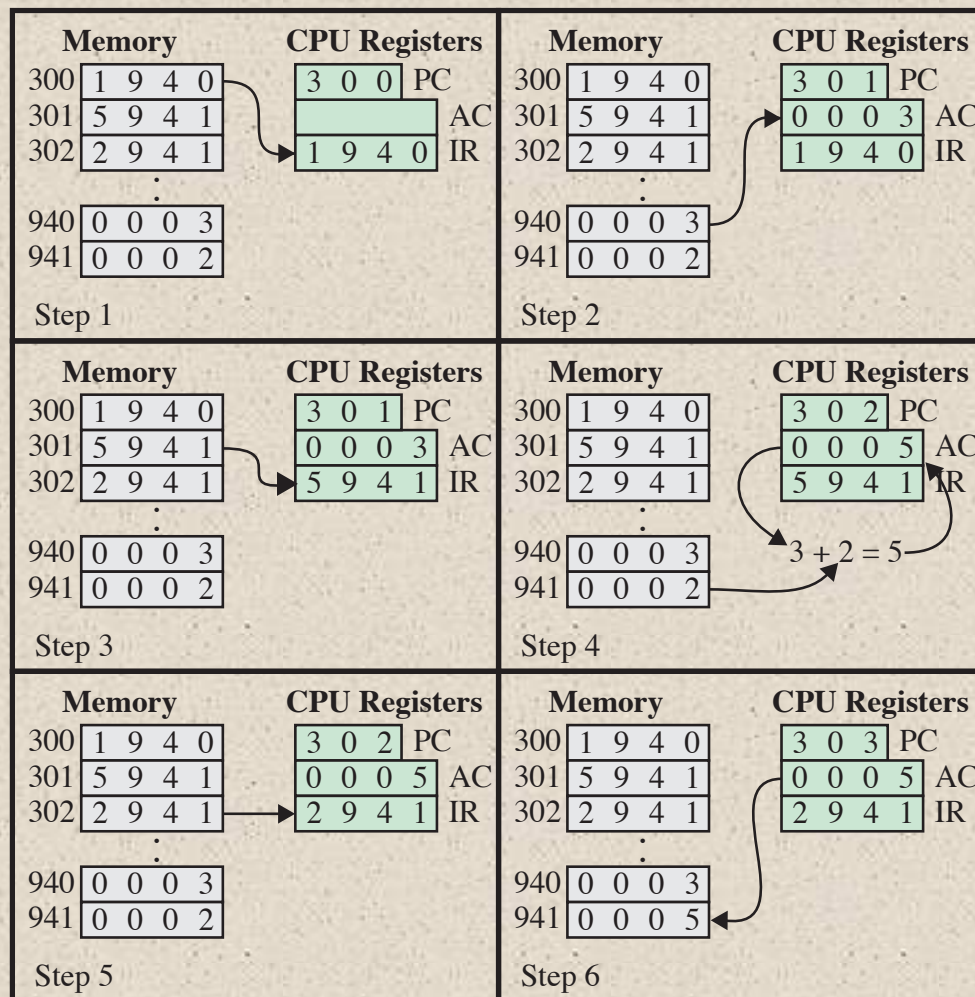
Program Counter (PC) = Address of instruction
Instruction Register (IR) = Instruction being executed
Accumulator (AC) = Temporary storage

(c) Internal CPU registers

0001 = Load AC from Memory
0010 = Store AC to Memory
0101 = Add to AC from Memory

(d) Partial list of opcodes

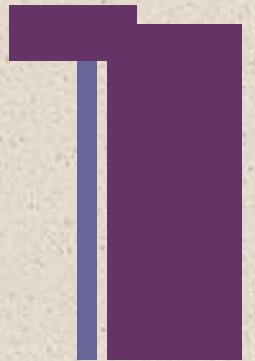
Figure 3.4 Characteristics of a Hypothetical Machine



**Figure 3.5 Example of Program Execution
(contents of memory and registers in hexadecimal)**



Example Program Execution



1. The PC contains 300, the address of the first instruction.
 - Value at address 300 is loaded (1940) into IR
 - First digit (1) is the op code (Load AC)
 - The last three digits are the address (940)
2. First instruction is implemented (0003 loaded to AC) and PC is incremented
3. Next instruction (address 301) is loaded (5941) into IR
 - First digit (5) is the op code (Add to AC)
 - The last three are the address (941)
4. The data at address 941 are added to the AC and the PC is incremented
5. Next instruction (address 302) is loaded (2941) into IR
 - First digit (2) is the op code (Store AC)
 - The last three are the address (941)
6. The contents of the AC are written to memory and the PC is incremented



Multiple Memory Accesses



- The execute cycle may involve more than one memory access.
- For example the PDP-11 includes the instruction ADD B,A
 - A PDP-11 has multiple registers rather than a single accumulator
- The instruction cycle would look like this: **Why such step?**
 - Fetch the instruction `ADD B,A`
 - Read the contents of memory location A into a register
 - Read the contents of memory location B into a different register
 - Add the two values
 - Write the result from the processor to memory location A
- This results in a more complicated execute cycle
- A more detailed instruction cycle reflects this

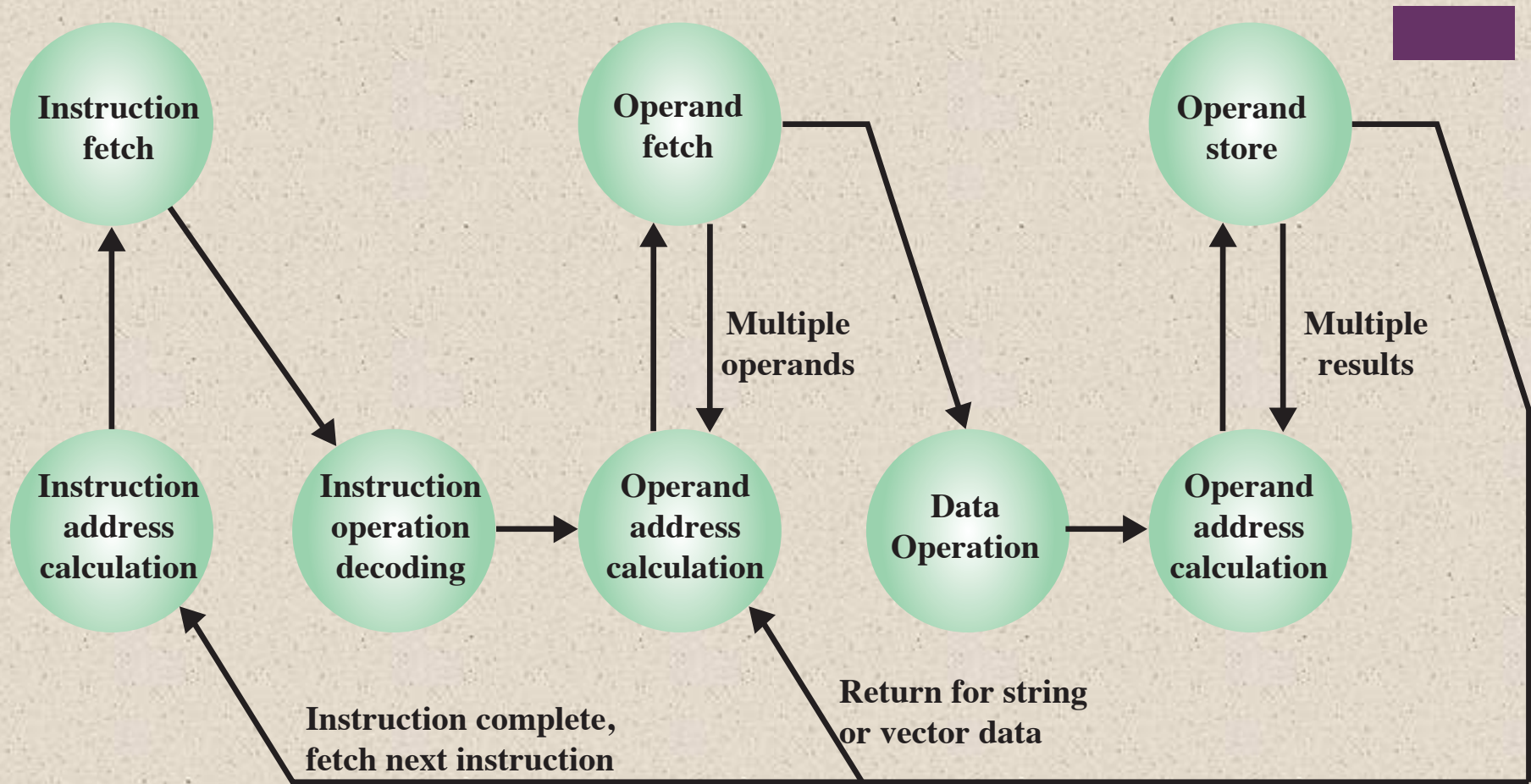


Figure 3.6 Instruction Cycle State Diagram



Interrupts

- An *interrupt* allows some module (I/O, memory) to stop the normal processing of the processor
 - It may or may not resume processing after handling the interrupt
- Interrupts are a way to improve processing efficiency
 - External devices are slower than the processor
 - We don't want the processor to wait for the printer
 - The printer can interrupt the processor when it needs attention
- Interrupts are also used to stop a process when some type of error has occurred
 - The error may or may not be recoverable

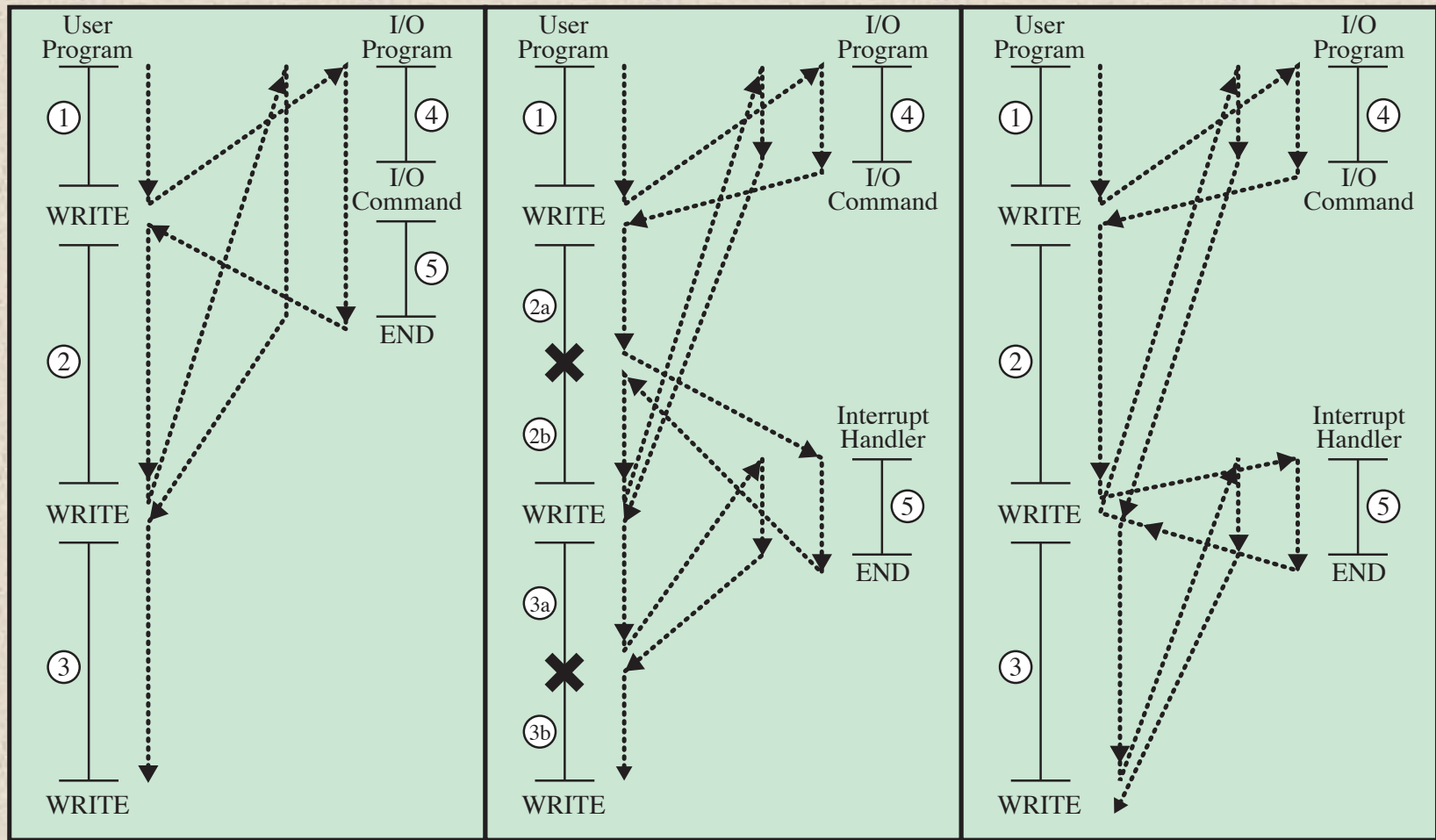
**Other way is
pooling --> simpler,
but less efficient**



Program	Generated by some condition that occurs as a result of an instruction execution, such as arithmetic overflow, division by zero, attempt to execute an illegal machine instruction, or reference outside a user's allowed memory space.
Timer	Generated by a timer within the processor. This allows the operating system to perform certain functions on a regular basis.
I/O	Generated by an I/O controller, to signal normal completion of an operation, request service from the processor, or to signal a variety of error conditions.
Hardware failure	Generated by a failure such as power failure or memory parity error.

Table 3.1

Classes of Interrupts



✕ = interrupt occurs during course of execution of user program

Figure 3.7 Program Flow of Control Without and With Interrupts

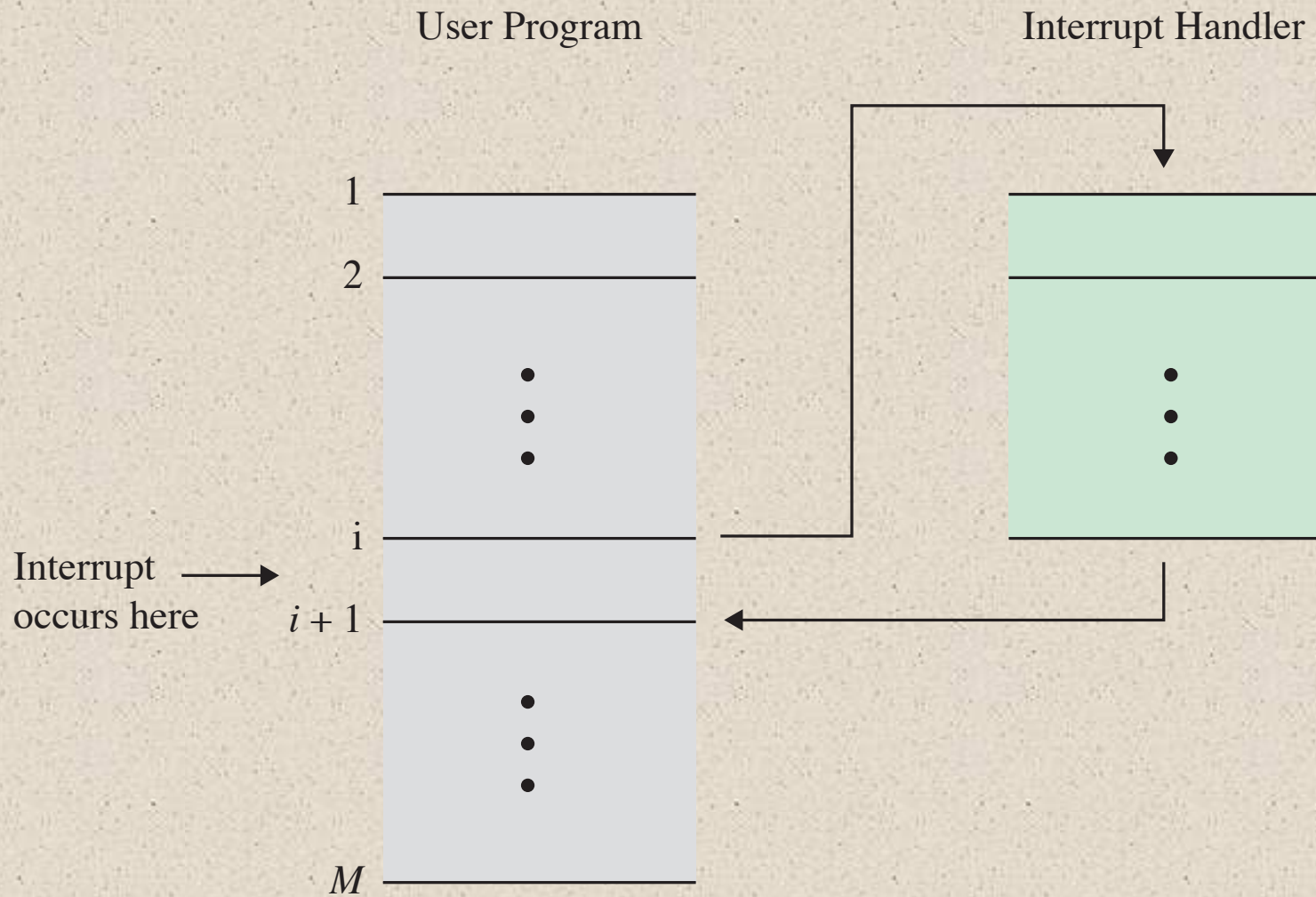


Figure 3.8 Transfer of Control via Interrupts

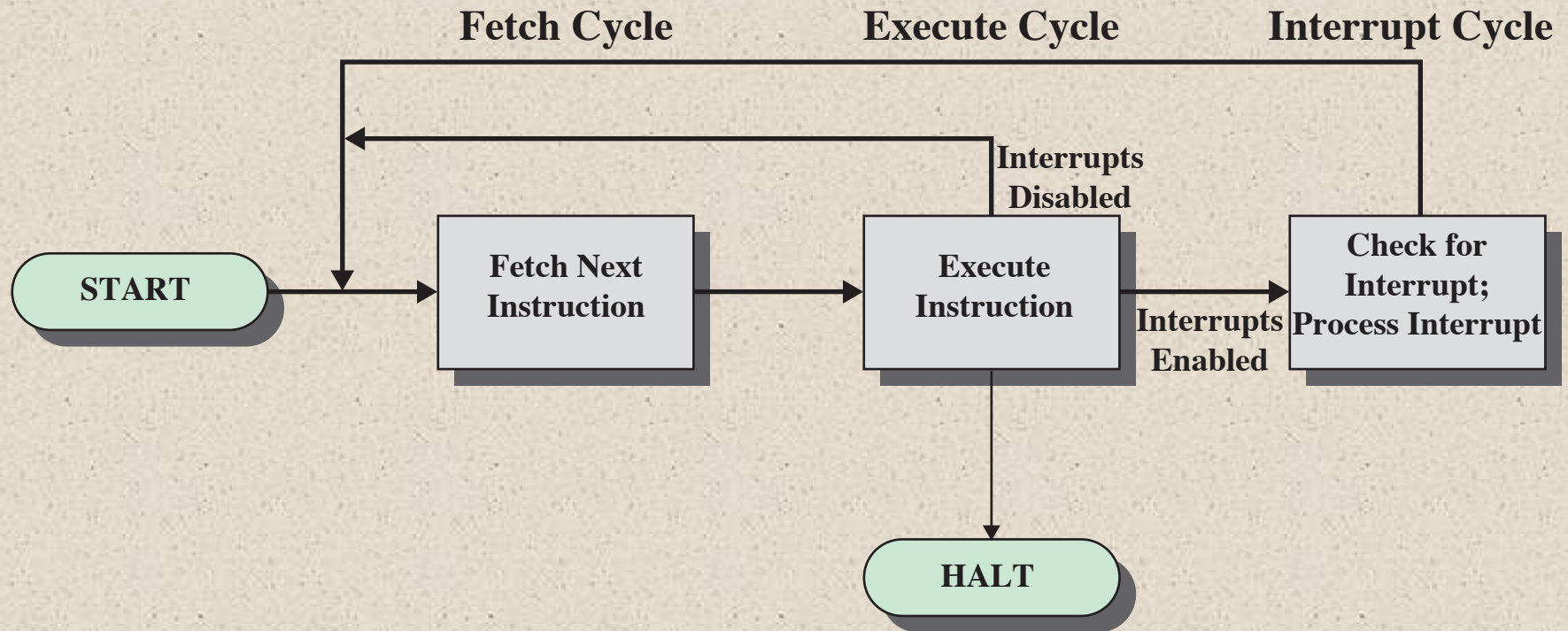


Figure 3.9 Instruction Cycle with Interrupts



Interrupts



- In these examples a “write” involves three steps:
 - Preprocessing
 - The write itself which takes place on a peripheral device
 - Postprocessing
- The first system, without interrupts, must wait during each write operation
- The second system, with interrupts, can initiate the write and then proceed with normal operations
 - When the write is complete the processor is interrupted to do the postprocessing necessary at the end of the write
- On the third system the processor tries to initiate a second write before the first is completed
 - Again, even with interrupts, the processor must wait

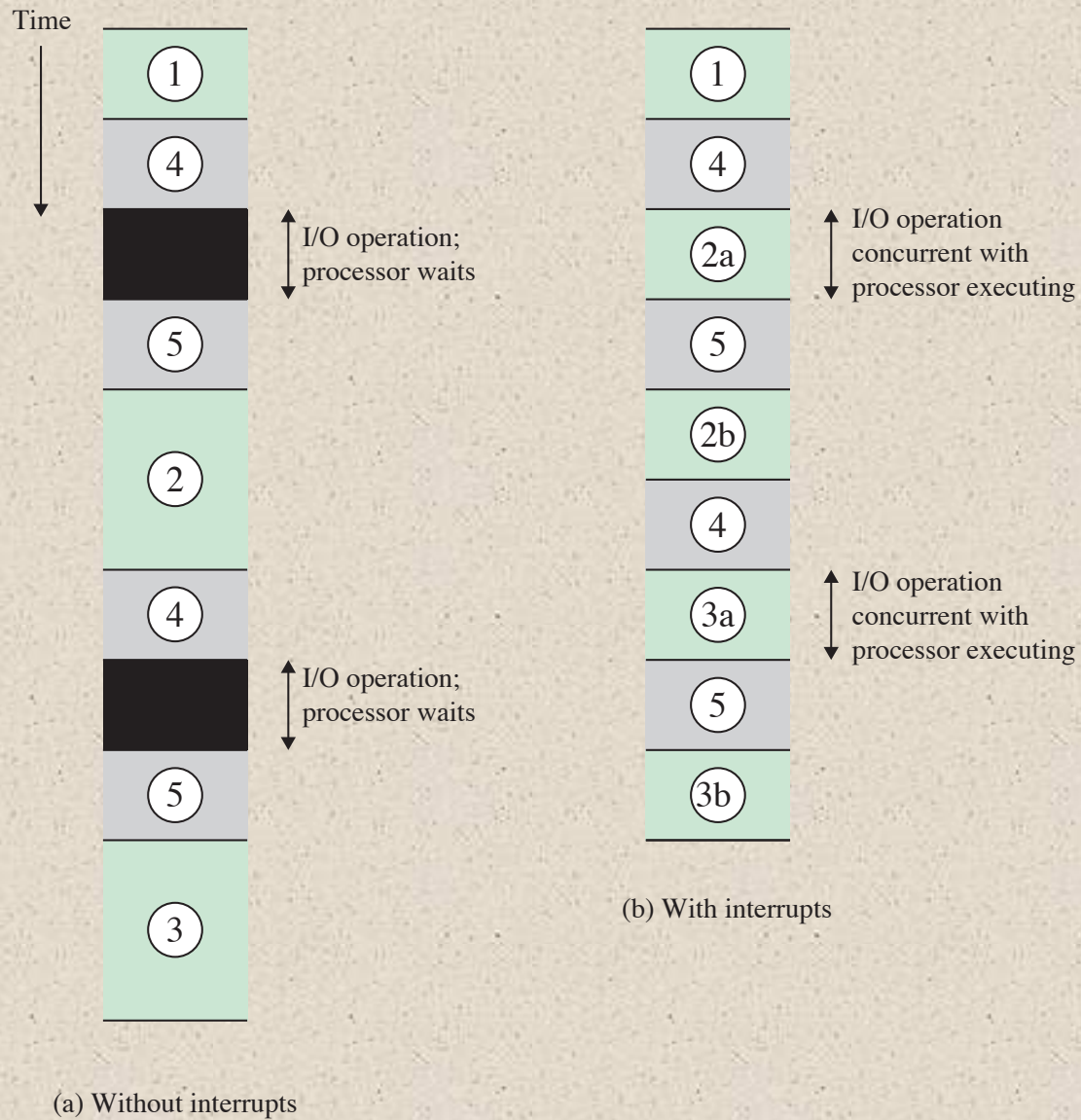


Figure 3.10 Program Timing: Short I/O Wait

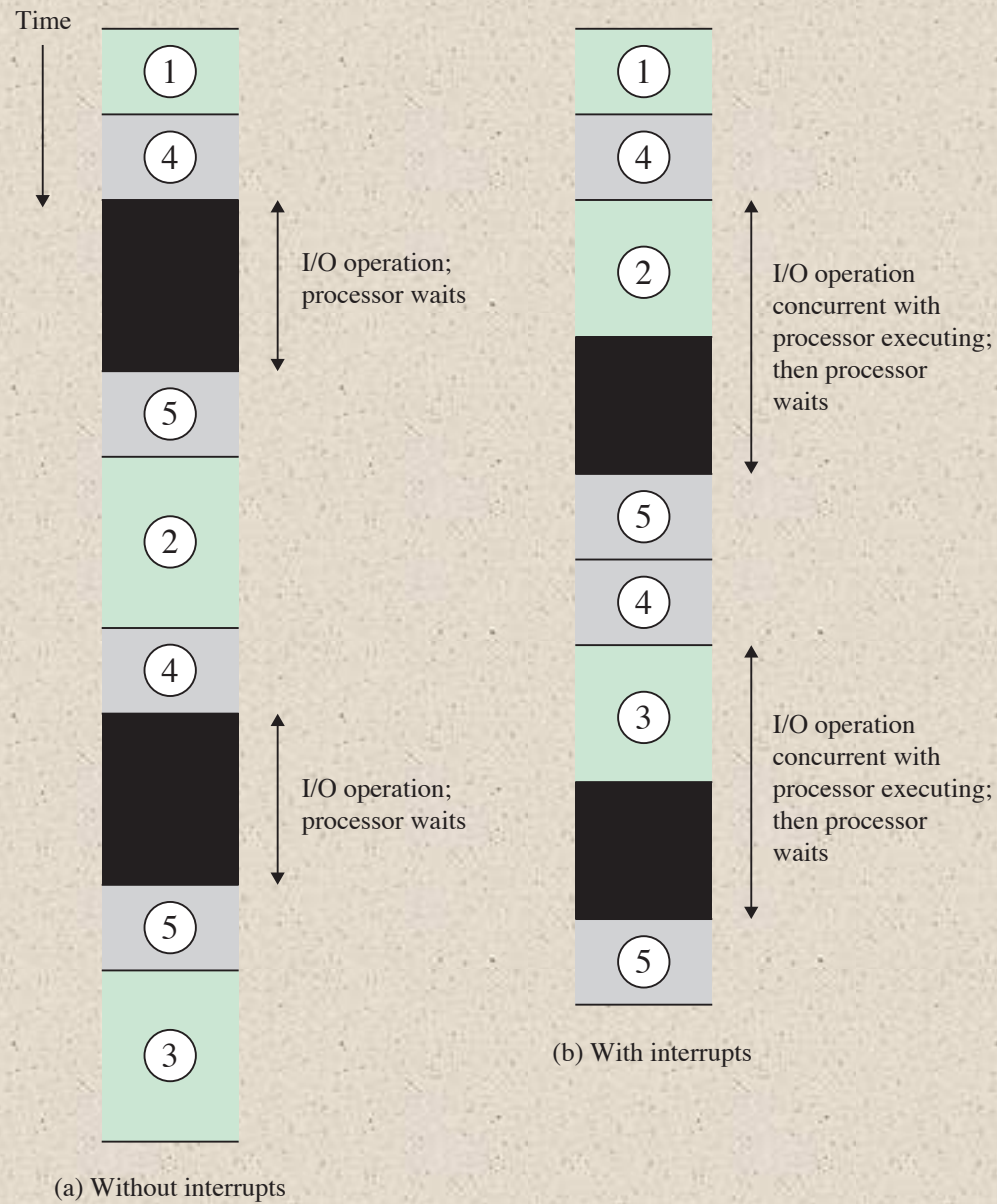


Figure 3.11 Program Timing: Long I/O Wait

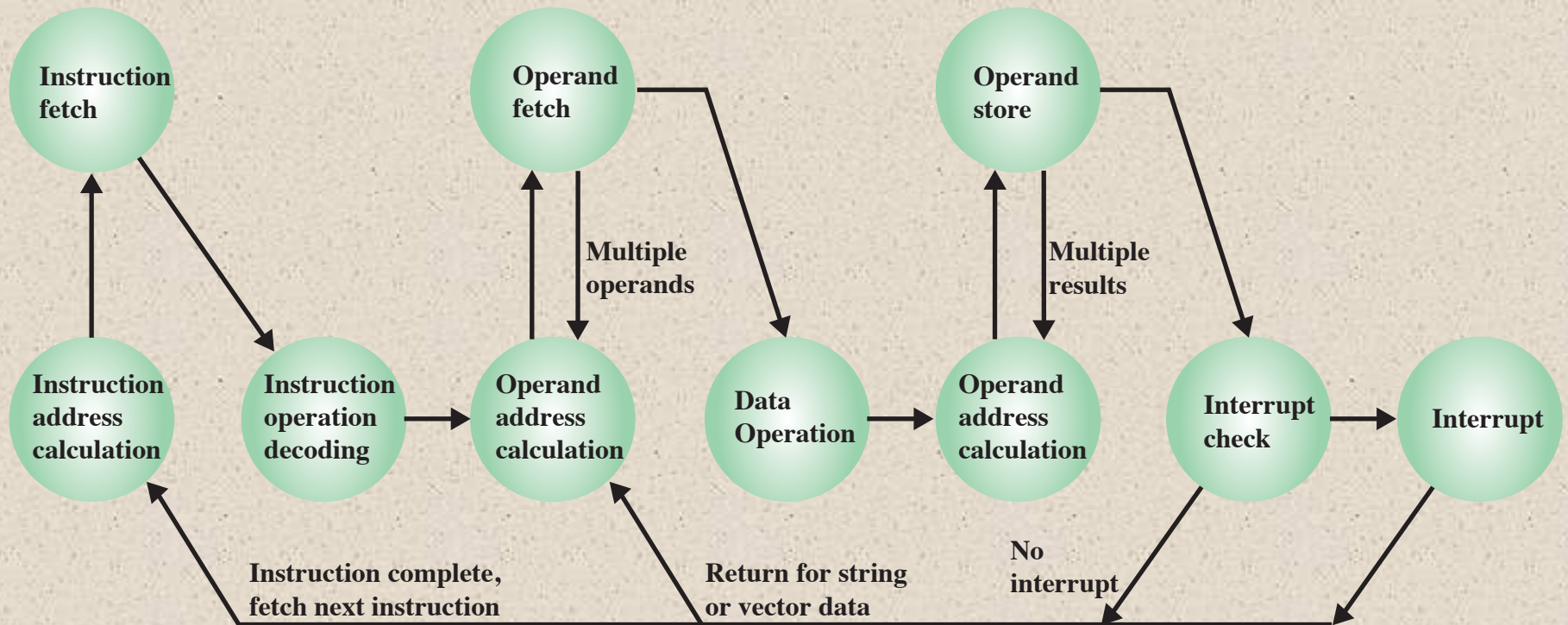
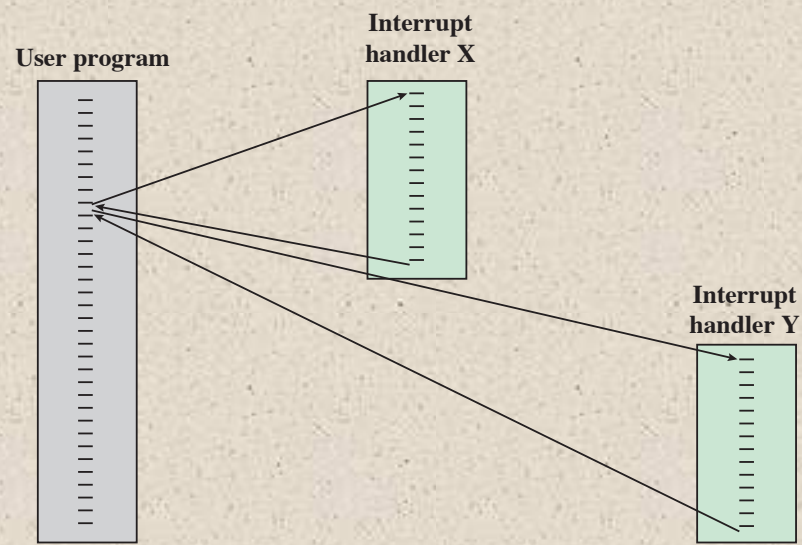


Figure 3.12 Instruction Cycle State Diagram, With Interrupts

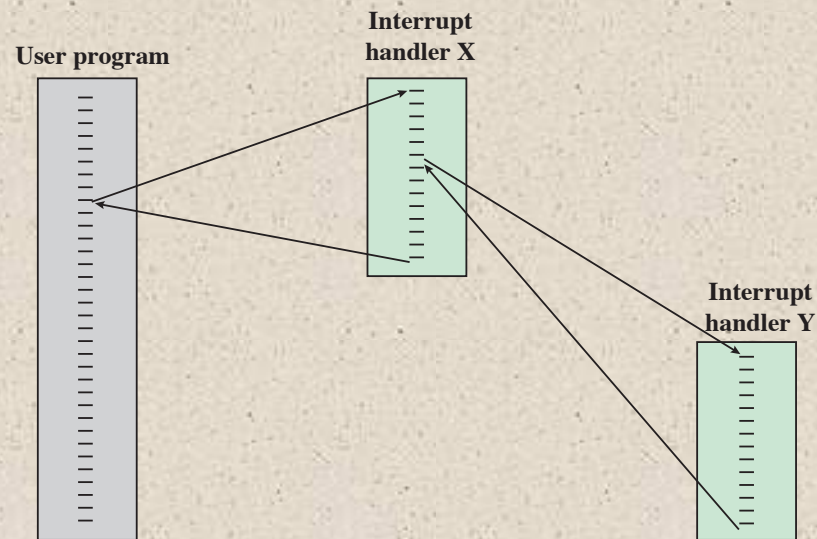


Multiple Interrupts

- Once interrupts are introduced, it become possible (likely?) that a interrupt will be thrown while the system is processing the handler for an earlier interrupt
- There are two strategies to handle this situation.
 1. Disabled interrupts – once the processor begins handling an interrupt, it will ignore further interrupts.
 - If a further interrupt occurs it will usually remain pending so it can be handled once the current handler completes
 - We need an interrupt queue
 - This approach does not take into account priority or time-critical needs
 2. Define interrupt priorities – a higher priority interrupt can interrupt the handler of one with a lower priority



(a) Sequential interrupt processing



(b) Nested interrupt processing

Figure 3.13 Transfer of Control with Multiple Interrupts



Priority Interrupts

- As an example of this second approach, consider a system with three I/O devices: a printer, a disk, and a communications line, with increasing priorities of 2, 4, and 5, respectively
 - A user program begins at $t = 0$
 - At $t = 10$, a printer interrupt occurs
 - While printer interrupt service routine (ISR) is still executing, at $t = 15$, a communications interrupt occurs.
 - Because the communications line has higher priority than the printer, the interrupt is honored. The printer ISR is interrupted
 - While this routine is executing, a disk interrupt occurs ($t = 20$)
 - Because this interrupt is of lower priority, it is simply held, and the communications ISR runs to completion.
 - When the communications ISR is complete ($t = 25$), the system processes the disk ISR because it has a higher priority
 - Only when that routine is complete ($t = 35$) is the printer ISR resumed.
 - When that routine completes ($t = 40$), control finally returns to the user program.

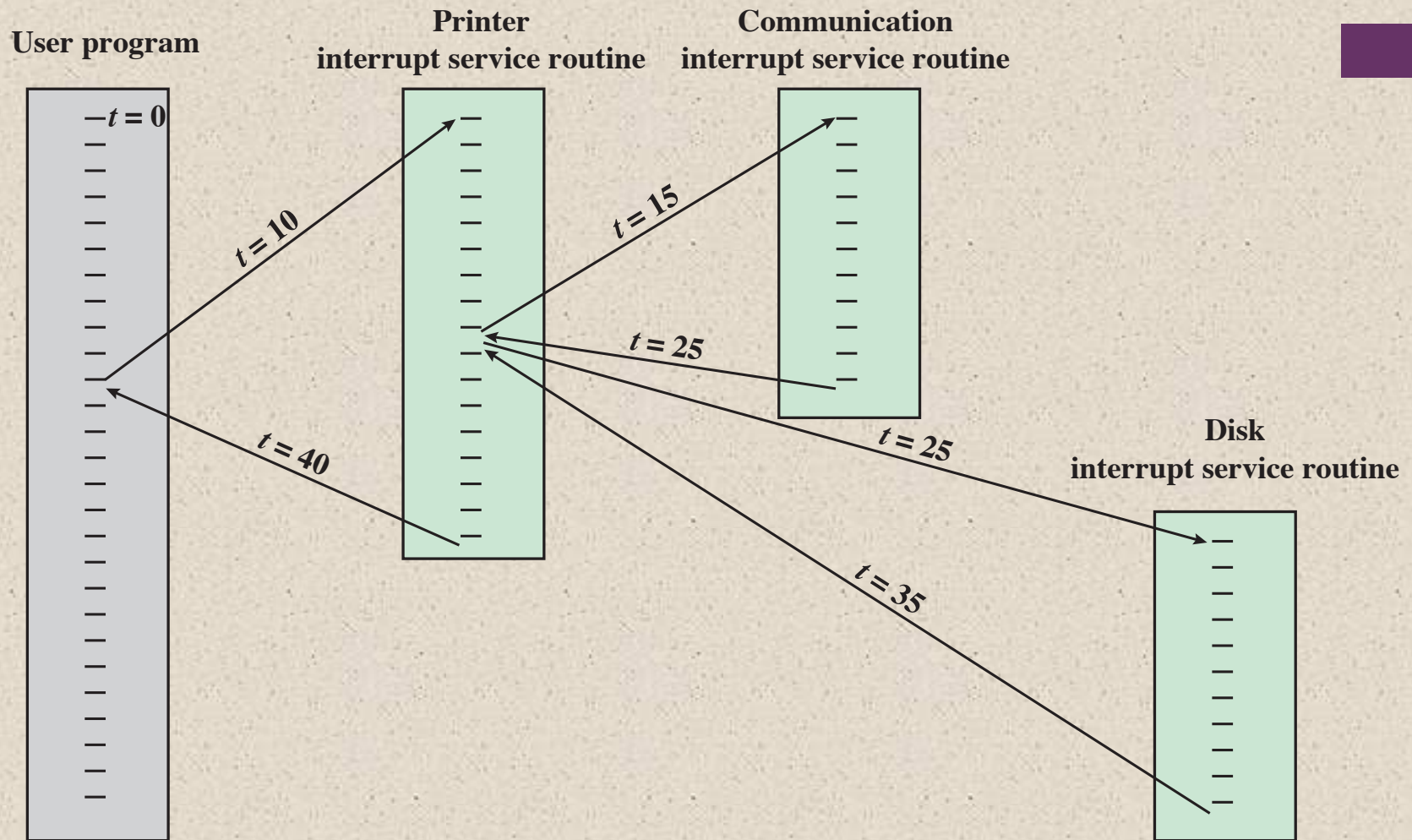


Figure 3.14 Example Time Sequence of Multiple Interrupts



I/O Function

- I/O module can exchange data directly with the processor
- Processor can read data from or write data to an I/O module
 - Processor identifies a specific device that is controlled by a particular I/O module
 - I/O instructions rather than memory referencing instructions
- In some cases it is desirable to allow I/O exchanges to occur directly with memory
 - The processor grants to an I/O module the authority to read from or write to memory so that the I/O memory transfer can occur without tying up the processor
 - The I/O module issues read or write commands to memory relieving the processor of responsibility for the exchange
 - This operation is known as direct memory access (DMA)

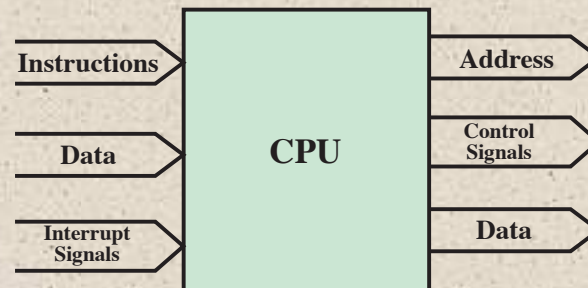
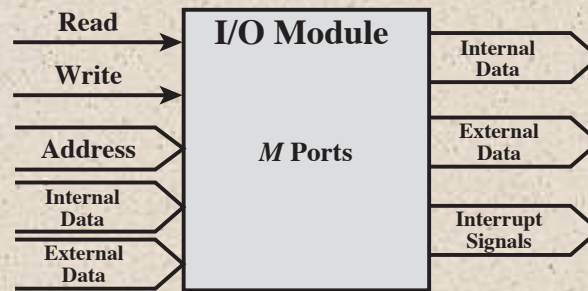
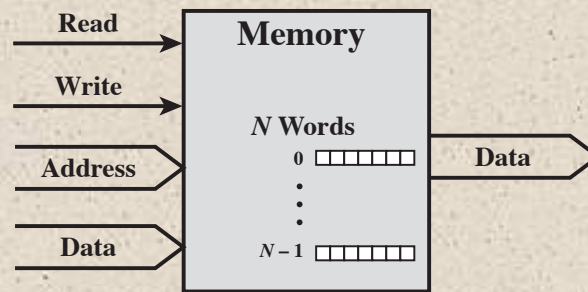
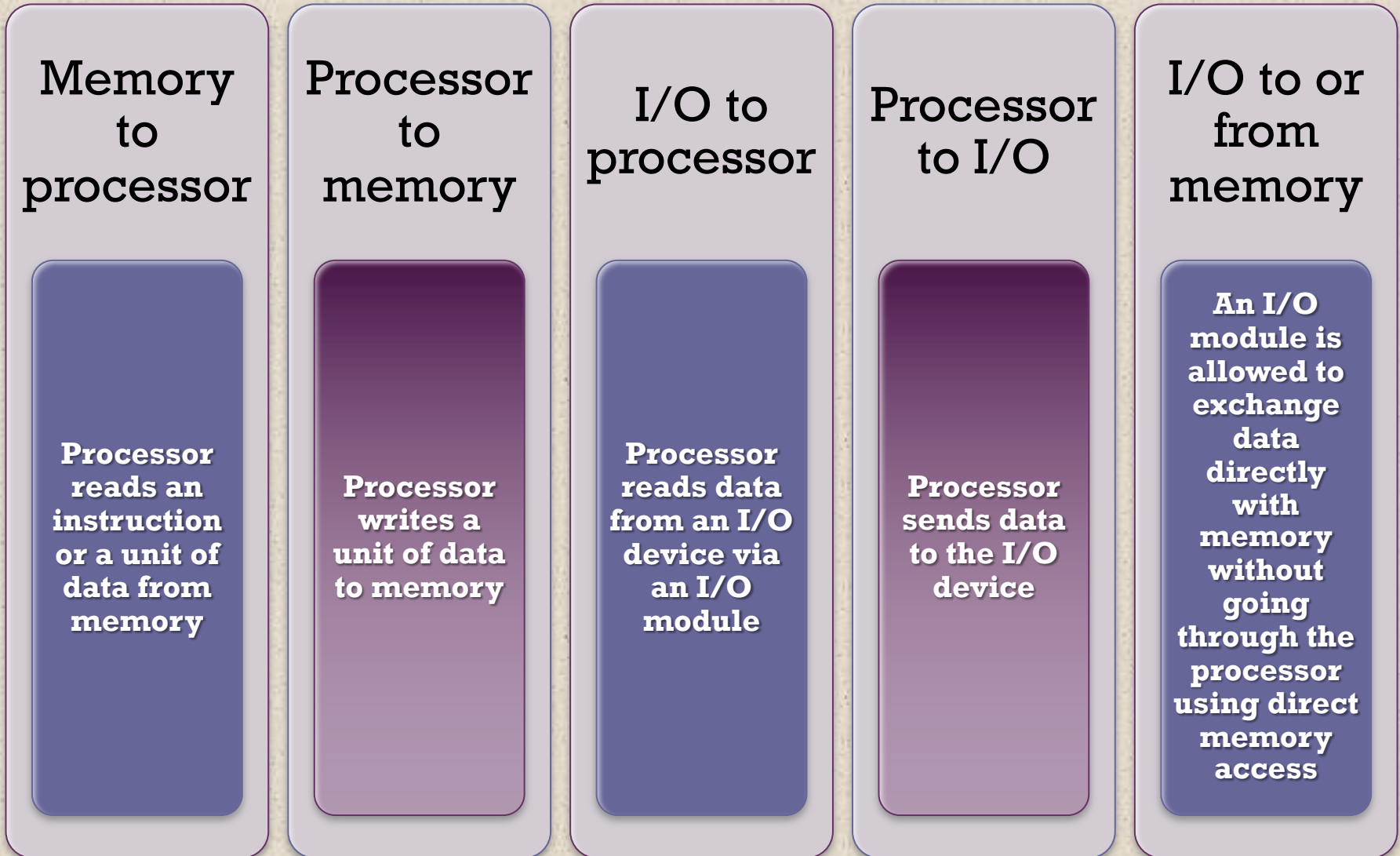
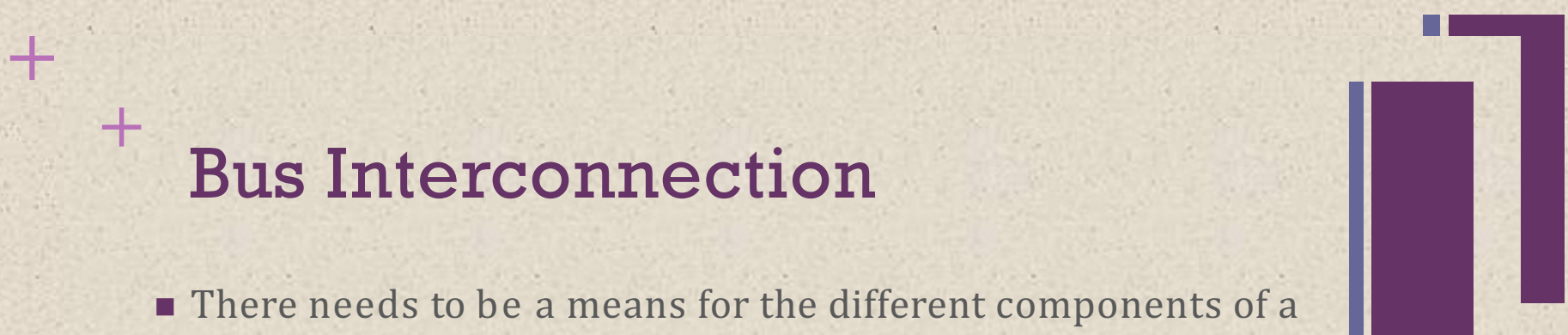


Figure 3.15 Computer Modules

The interconnection structure must support the following types of transfers:





Bus Interconnection

- There needs to be a means for the different components of a computer system to communicate
- Modern general-purpose computers use various specialized point-to-point interconnection structures
- A common bus structure was previously dominant and is still used for many embedded systems
 - Multiple devices connect to the bus
 - A signal transmitted by one device can be received by any device on the bus
 - The devices must be synchronized so that signals do not overlap

A communication pathway connecting two or more devices

- Key characteristic is that it is a shared transmission medium

Signals transmitted by any one device are available for reception by all other devices attached to the bus

- If two devices transmit during the same time period their signals will overlap and become garbled



Typically consists of multiple communication lines

- Each line is capable of transmitting signals representing binary 1 and binary 0

Computer systems contain a number of different buses that provide pathways between components at various levels of the computer system hierarchy



System bus

- A bus that connects major computer components (processor, memory, I/O)

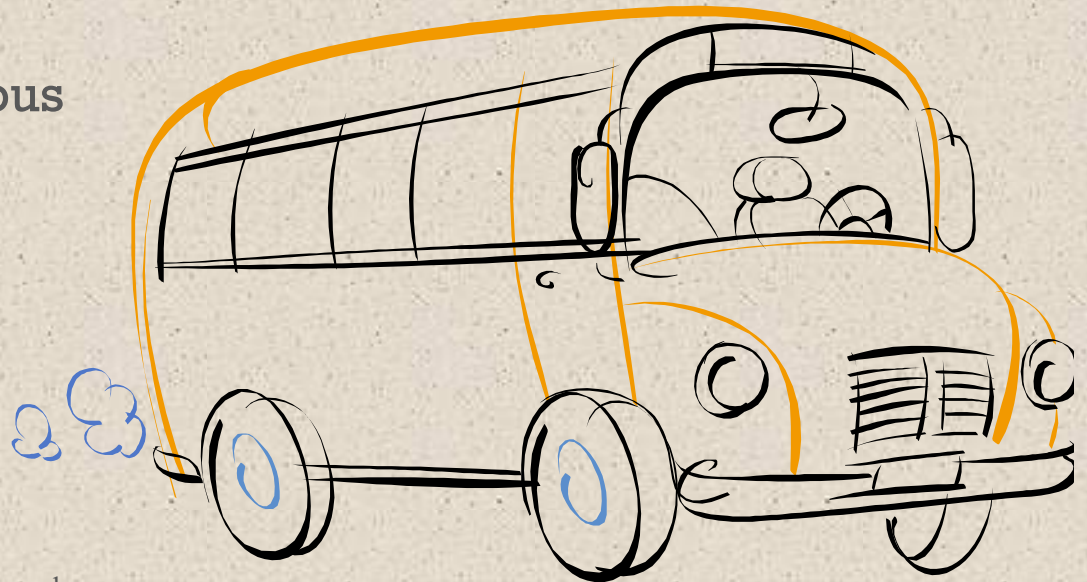
The most common computer interconnection structures are based on the use of one or more system buses

B
u
s

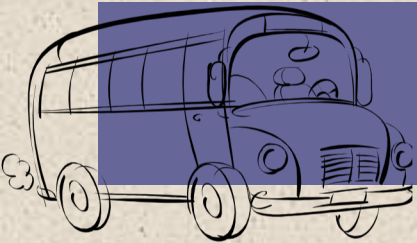
I
n
t
e
r
c
o
n

Data Bus

- Data lines that provide a path for moving data among system modules
- May consist of 32, 64, 128, or more separate lines
- The number of lines is referred to as the *width* of the data bus
- The number of lines determines how many bits can be transferred at a time
- The width of the data bus is a key factor in determining overall system performance

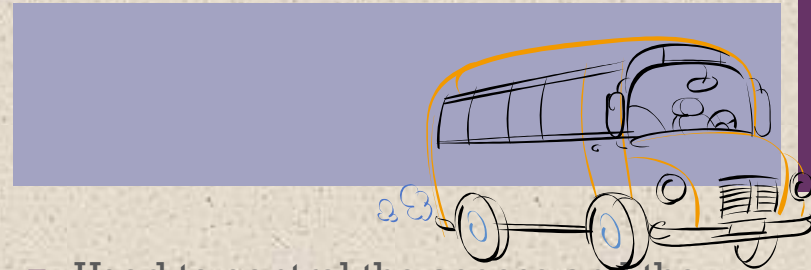


+ Address Bus



- Used to designate the source or destination of the data on the data bus
 - If the processor wishes to read a word of data from memory it puts the address of the desired word on the address lines
- Width determines the maximum possible memory capacity of the system
- Also used to address I/O ports
 - The higher order bits are used to select a particular module on the bus and the lower order bits select a memory location or I/O port within the module

Control Bus



- Used to control the access and the use of the data and address lines
- Because the data and address lines are shared by all components there must be a means of controlling their use
- Control signals transmit both command and timing information among system modules
- Timing signals indicate the validity of data and address information
- Command signals specify operations to be performed

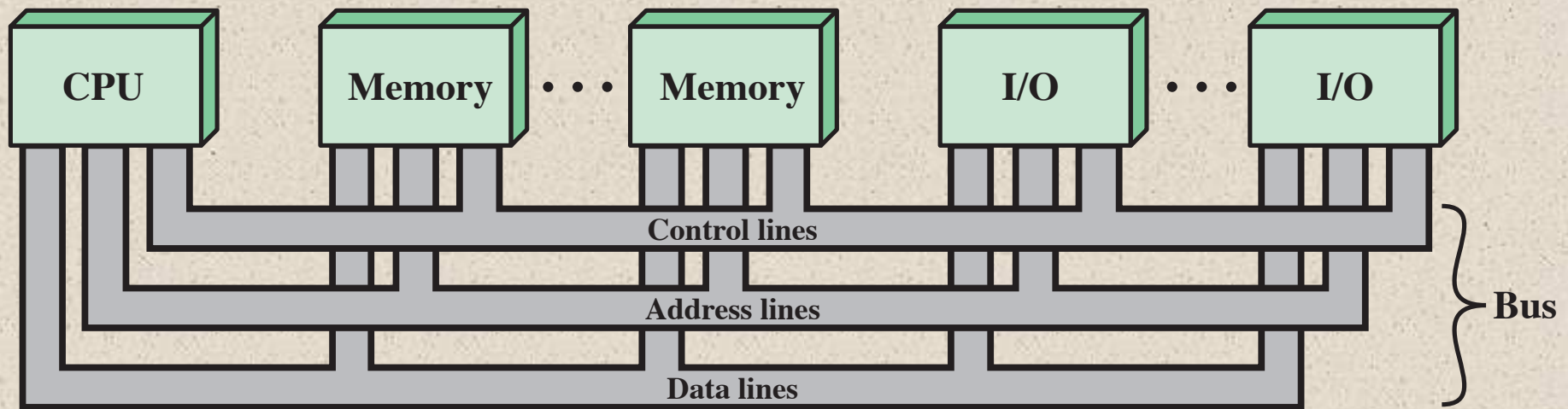


Figure 3.16 Bus Interconnection Scheme



Point-to-Point Interconnect



Principal reason for change was the electrical constraints encountered with increasing the frequency of wide synchronous buses

At higher and higher data rates it becomes increasingly difficult to perform the synchronization and arbitration functions in a timely fashion

A conventional shared bus on the same chip magnified the difficulties of increasing bus data rate and reducing bus latency to keep up with the processors

Has lower latency, higher data rate, and better scalability

+ Quick Path Interconnect

- Introduced in 2008
- Multiple direct connections
 - Direct pairwise connections to other components eliminating the need for arbitration found in shared transmission systems
- Layered protocol architecture
 - These processor level interconnects use a layered protocol architecture rather than the simple use of control signals found in shared bus arrangements
- Packetized data transfer
 - Data are sent as a sequence of packets each of which includes control headers and error control codes

QPI



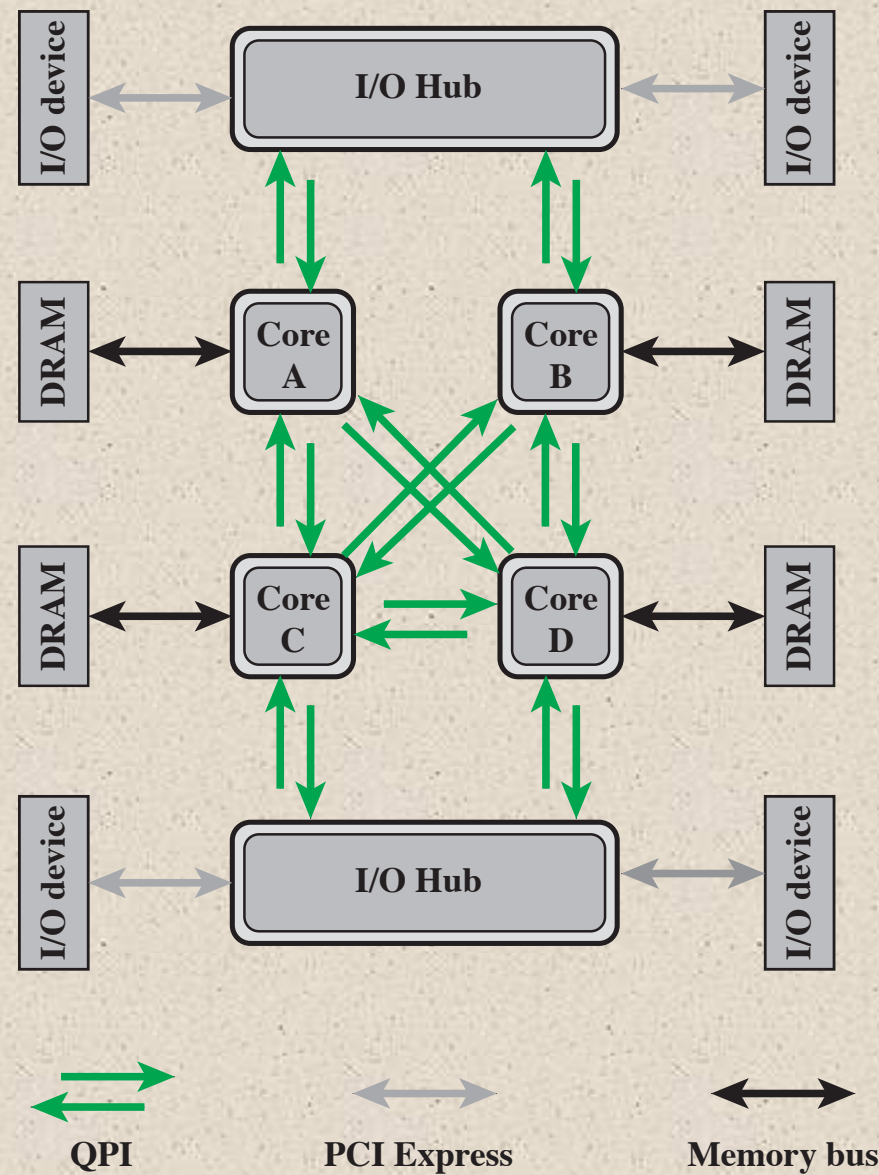


Figure 3.17 Multicore Configuration Using QPI



QPI Layers



QPI is defined as a four-layer protocol architecture:

- **Physical Layer:** the actual wires carrying signals, along with the circuitry and logic to support the transfer of 1's and 0's.
 - The unit of data transfer at the physical layer is 20 bits, which is called a Phit (physical unit)
- **Link Layer:** responsible for reliable transmission and flow control.
 - The link layer's unit of transfer is an 80 bit Flit (flow control unit)
- **Routing Layer:** Provides the framework for directing packets.
- **Protocol Layer:** The high-level set of rules for exchanging packets of data between devices. A packet is an integral number of Flits.

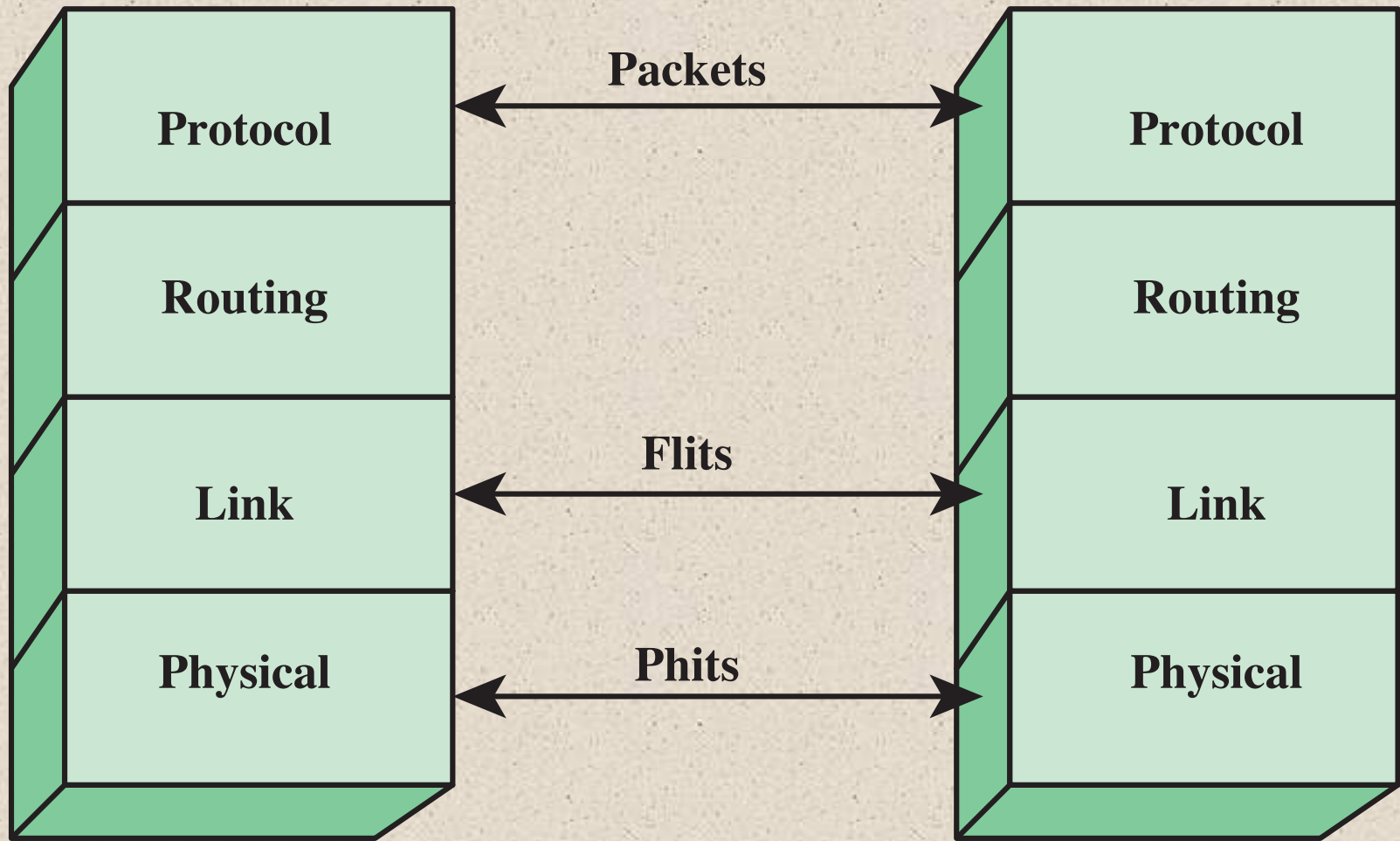


Figure 3.18 QPI Layers

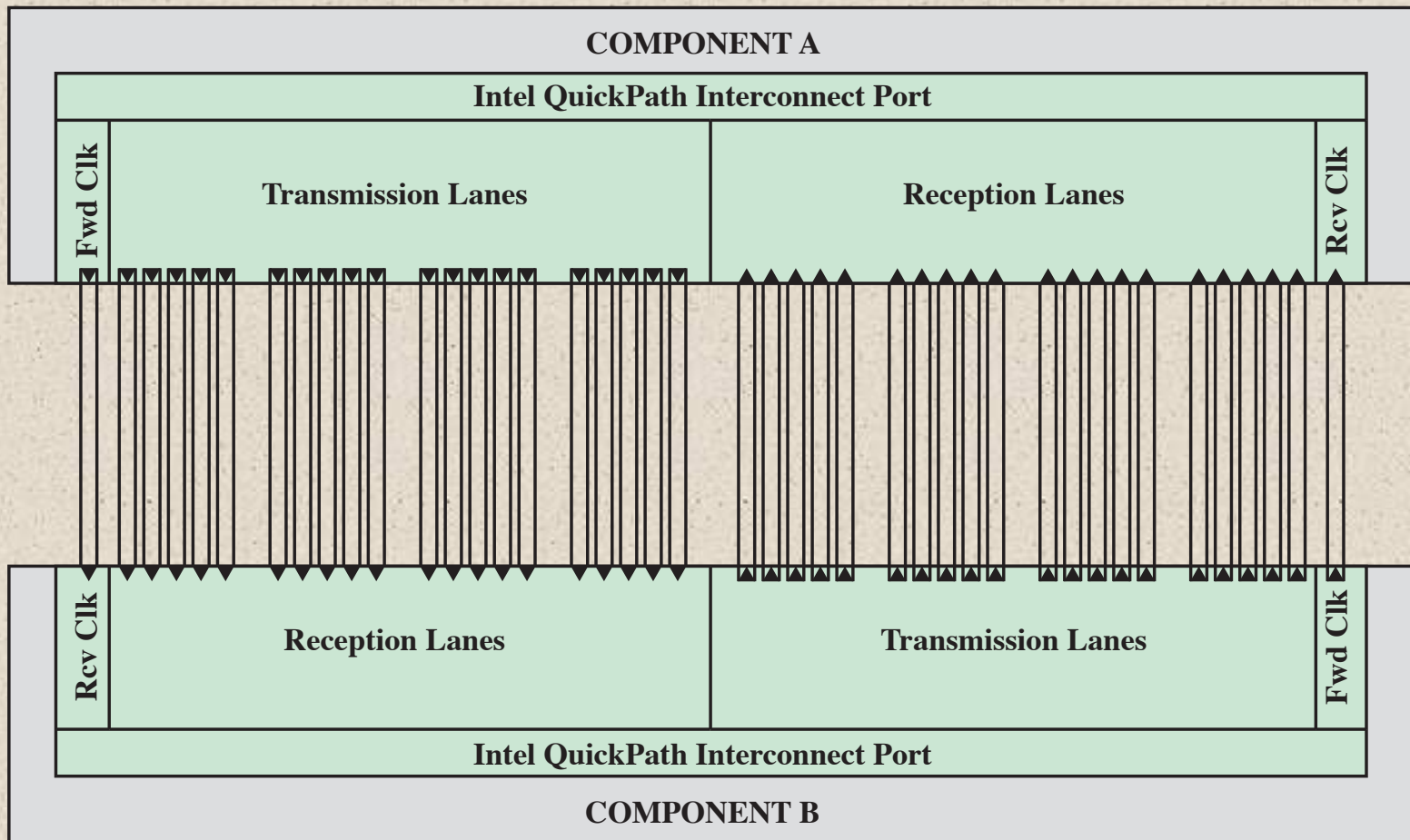


Figure 3.19 Physical Interface of the Intel QPI Interconnect



QPI Physical Layer

- The previous figure shows the physical architecture of a QPI port
- The QPI port consists of 84 individual links grouped as follows
 - Each data path consists of a pair of wires that transmits data one bit at a time; each pair is referred to as a **lane**
 - There are 20 data lanes in each direction (transmit and receive), plus a clock lane in each direction
 - Thus, QPI is capable of transmitting 20 bits in parallel in each direction
 - The 20-bit unit is referred to as a *Phit*
 - The form of transmission on each lane is known as **differential signaling, or balanced transmission** – signals are transmitted as a current that travels down one conductor and returns on the other
 - The binary value depends on the voltage difference
 - Typically, one line has a positive voltage value and the other line has zero voltage, and one line is associated with binary 1 and one line is associated with binary 0



QPI Physical Layer Lanes

- Another function performed by the physical layer is that it manages the translation between 80-bit flits and 20-bit phits using a technique known as **multilane distribution**
- The flits can be considered as a bit stream that is distributed across the data lanes in a round-robin fashion (first bit to first lane, second bit to second lane, etc.)
- This approach enables QPI to achieve very high data rates by implementing the physical link between two ports as multiple parallel channels.

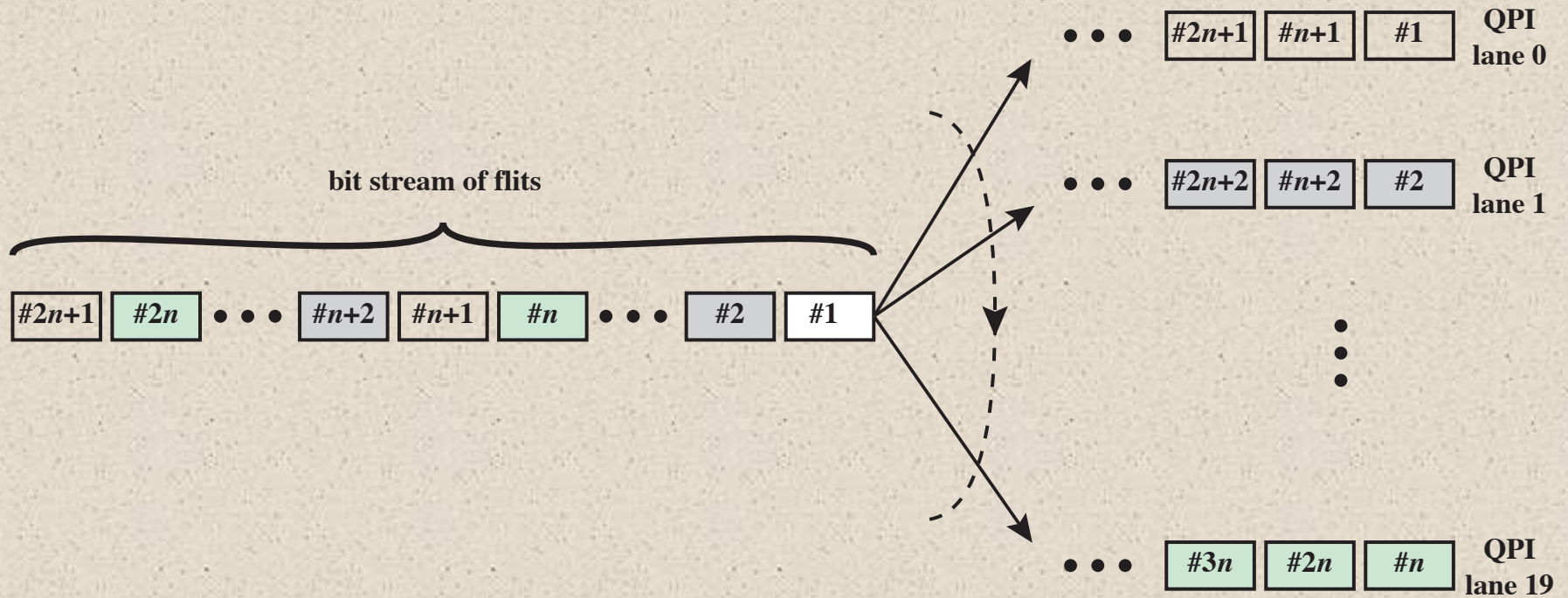


Figure 3.20 QPI Multilane Distribution



QPI Link Layer



- Performs two key functions: *flow control* and *error control*
 - Operate on the level of the flit (flow control unit)
 - Each flit consists of a 72-bit message payload and an 8-bit error control code called a *cyclic redundancy check* (CRC)
- Flow control function
 - Needed to ensure that a sending QPI entity does not overwhelm a receiving QPI entity by sending data faster than the receiver can process the data and clear buffers for more incoming data
- Error control function
 - Detects and recovers from bit errors, and so isolates higher layers from experiencing bit errors



QPI Routing and Protocol Layers

Routing Layer

- Used to determine the course that a packet will traverse across the available system interconnects
- Defined by firmware and describe the possible paths that a packet can follow

Protocol Layer

- Packet is defined as the unit of transfer
- One key function performed at this level is a cache coherency protocol which deals with making sure that main memory values held in multiple caches are consistent
- A typical data packet payload is a block of data being sent to or from a cache



Peripheral Component Interconnect (PCI)



- A popular high bandwidth, processor independent bus that can function as a mezzanine or peripheral bus
- Delivers better system performance for high speed I/O subsystems
- PCI Special Interest Group (SIG)
 - Created to develop further and maintain the compatibility of the PCI specifications
- PCI Express (PCIe)
 - Point-to-point interconnect scheme intended to replace bus-based schemes such as PCI
 - Key requirement is high capacity to support the needs of higher data rate I/O devices, such as Gigabit Ethernet
 - Another requirement deals with the need to support time dependent data streams

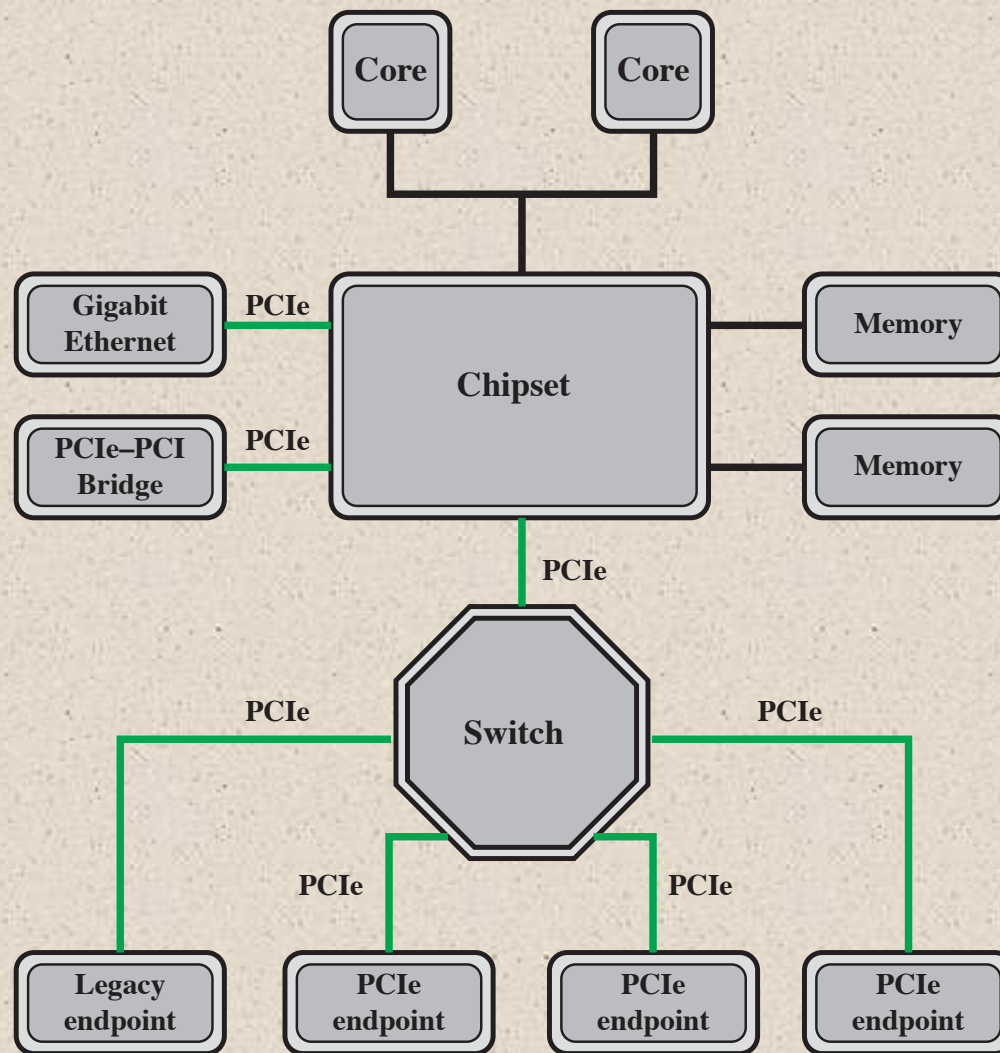
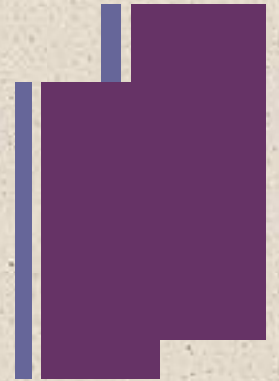


Figure 3.21 Typical Configuration Using PCIe



PCIe Layers



PCIe is described by a three layer architecture

- Physical Layer – the physical wires that carry the data
 - Also includes the circuitry and logic to support the transmission and receipt of 1's and 0's
- Data Link Layer – responsible for reliable transmission and flow control
 - Data packets at the data link layer are called DLLPs
- Transaction Layer – generates and consumes data packets and manages the flow between two components
 - Data packets at the transaction layer are called TLPs

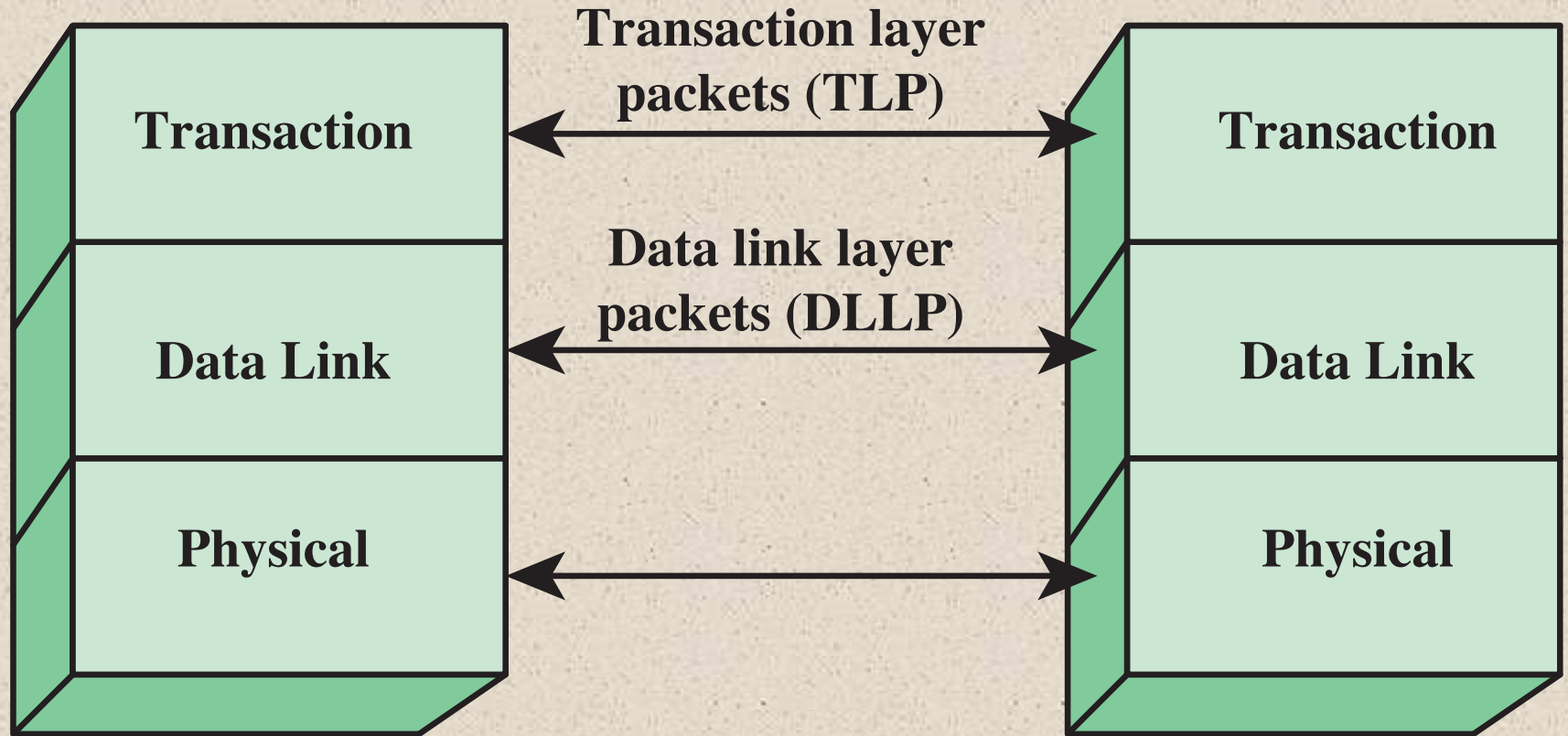
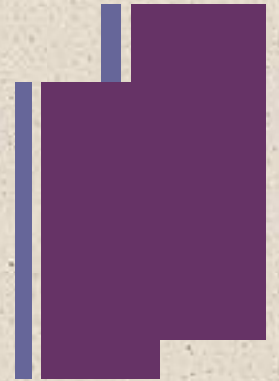


Figure 3.22 PCIe Protocol Layers

+ + PCIe Physical Layer



- PCIe operates in a similar manner to QPI with the exception that lanes are bidirectional and the number of lanes on a PCIe port can be 1, 4, 6, 16, or 32
- Bits sent to lanes in a round-robin scheme
- At each physical lane data are buffered and processed 16 bytes (128 bits) at a time
- Each block is encoded into a 130 bit code word for transmission
- There is no common clock; the receiver looks for transitions in the data for synchronization
- The extra 2 bits ensure that in a long sequence of 1's there are at least some 0's to provide these transitions

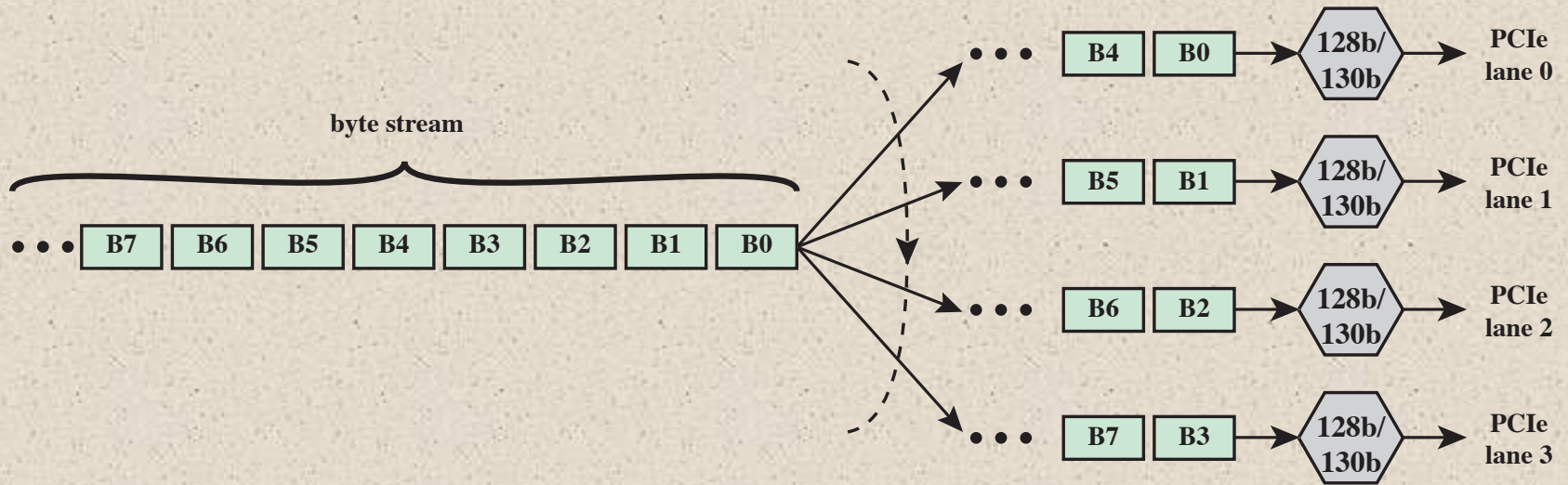


Figure 3.23 PCIe Multilane Distribution

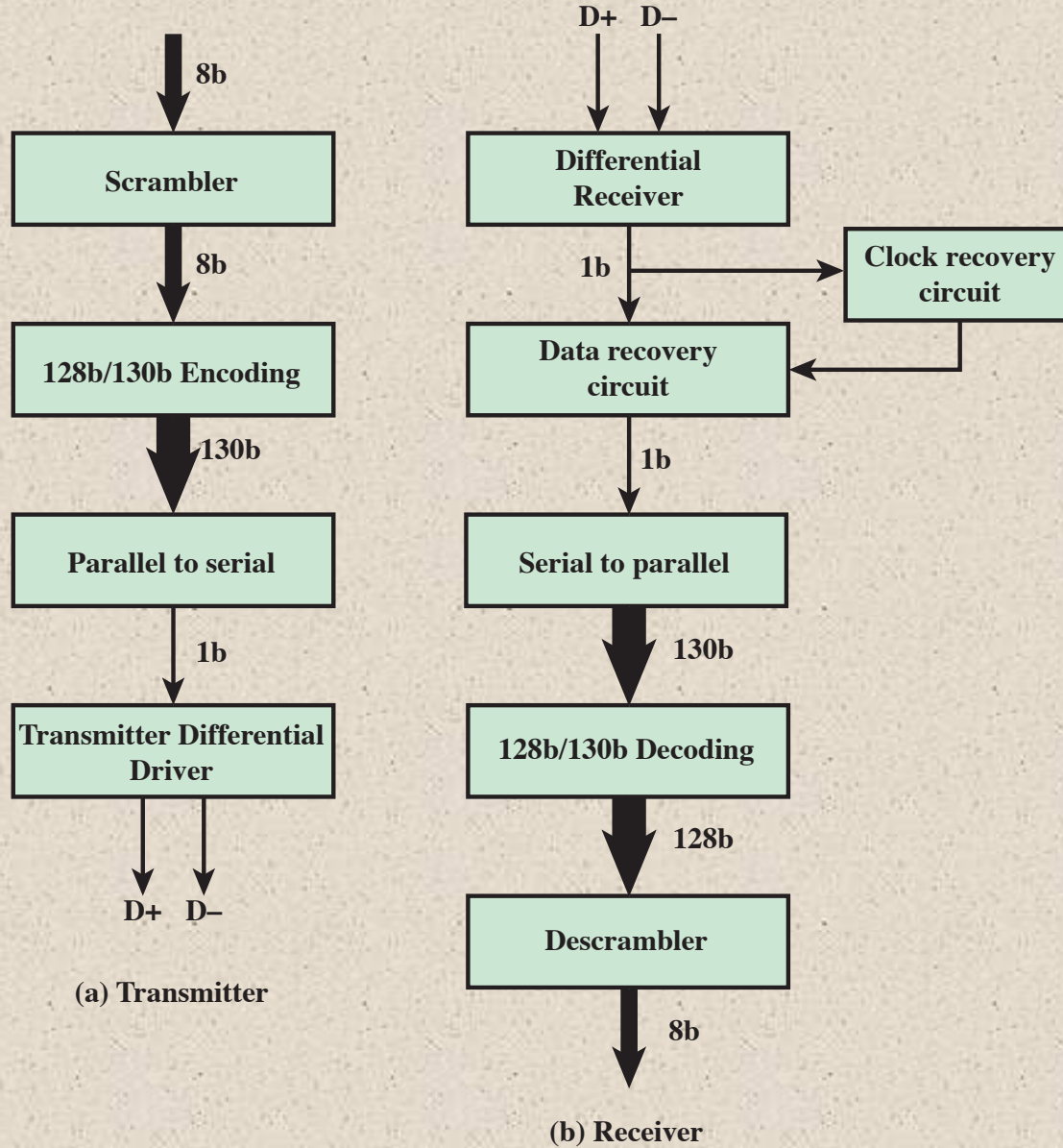
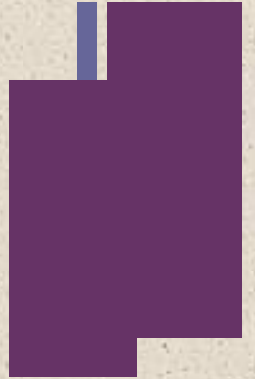


Figure 3.24 PCIe Transmit and Receive Block Diagrams

+ + PCIe Data Link Layer



- The data link layer sends packets between two devices that are concerned with “bookkeeping” between the two devices.
- The data link layer also adds error checking and address bits to each TLP to ensure they arrive at the correct device
- These packets fall into the following categories:
 - Flow control packets – regulate the rate at which TLPs and DLLPs can be transmitted
 - Power management packets – manage power platform budgeting
 - ACK packets to acknowledge the receipt of a valid data packet
 - NAK packets acknowledges the receipt of an invalid data packet – the packet must be resent



PCIe

Transaction Layer (TL)



- Receives read and write requests from the software above the TL and creates request packets for transmission to a destination via the link layer
- Most transactions use a *split transaction* technique
 - A request packet is sent out by a source PCIe device which then waits for a response called a *completion* packet
- TL messages and some write transactions are posted transactions (meaning that no response is expected)
- TL packet format supports 32-bit memory addressing and extended 64-bit memory addressing



The TL supports four address spaces:

■ Memory

- The memory space includes system main memory and PCIe I/O devices
- Certain ranges of memory addresses map into I/O devices

■ Configuration

- This address space enables the TL to read/write configuration registers associated with I/O devices

■ I/O

- This address space is used for legacy PCI devices, with reserved address ranges used to address legacy I/O devices

■ Message

- This address space is for control signals related to interrupts, error handling, and power management

Table 3.2

PCIe TLP Transaction Types

Address Space	TLP Type	Purpose
Memory	Memory Read Request	Transfer data to or from a location in the system memory map.
	Memory Read Lock Request	
	Memory Write Request	
I/O	I/O Read Request	Transfer data to or from a location in the system memory map for legacy devices.
	I/O Write Request	
Configuration	Config Type 0 Read Request	Transfer data to or from a location in the configuration space of a PCIe device.
	Config Type 0 Write Request	
	Config Type 1 Read Request	
	Config Type 1 Write Request	
Message	Message Request	Provides in-band messaging and event reporting.
	Message Request with Data	
Memory, I/O, Configuration	Completion	Returned for certain requests.
	Completion with Data	
	Completion Locked	
	Completion Locked with Data	

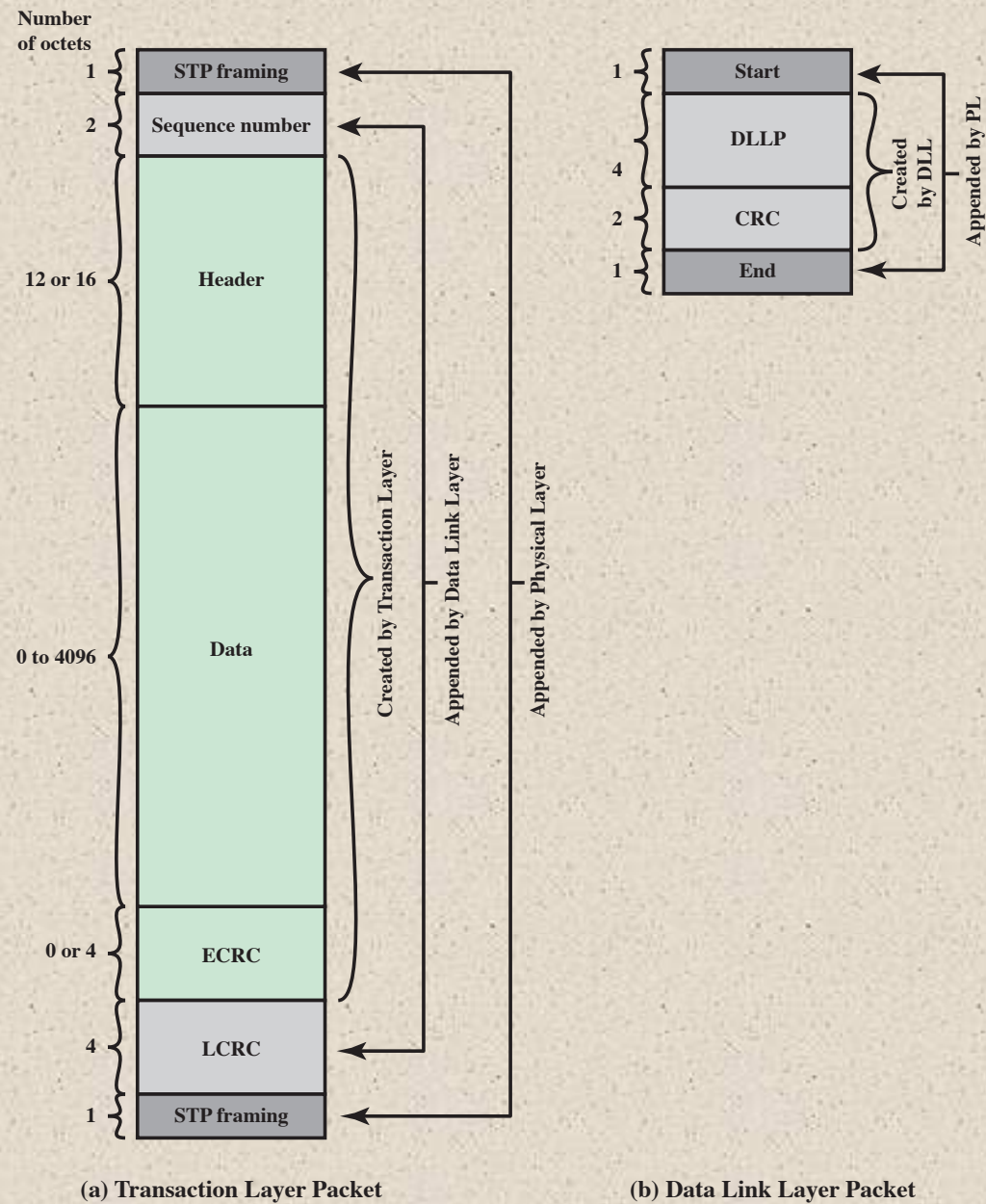


Figure 3.25 PCIe Protocol Data Unit Format

+ Summary

Chapter 3

- Computer components
- Computer function
 - Instruction fetch and execute
 - Interrupts
 - I/O function
- Interconnection structures
- Bus interconnection

A Top-Level View of Computer Function and Interconnection

- Point-to-point interconnect
 - QPI physical layer
 - QPI link layer
 - QPI routing layer
 - QPI protocol layer
- PCI express
 - PCI physical and logical architecture
 - PCIe physical layer
 - PCIe transaction layer
 - PCIe data link layer